



DEBRE BIRHAN UNIVERSITY
COLLEGE OF COMPUTING
DEPARTMENT OF INFORMATION SYSTEM

Emare E-Learning Management System

A Final Year Project Report Submitted to the Department of Information System in Partial Fulfillment of the Requirements for the Bachelor of Science Degree in Information System

Submitted By:

GROUP MEMBER	ID
Amen Terefe	1601405
Ayires Zebene	1601409
Asamnew Agiz	1601394

Date: June 2026

[Advisor Full Name], [Emare E-Learning Management System]

Submitted to: Emare ICT Hub, Debre Birhan, Ethiopia

DECLARATION

I hereby declare that this project report titled "Emare E-Learning Management System" is my own original work and has not been presented for a degree in any other university. All sources of information used in this project have been duly acknowledged and properly cited according to IEEE referencing standards.

I also declare that this work complies with the academic integrity standards of Debre Birhan University, College of Computing, Department of Information System.

Student Name: Amen Terefe

Student Name: Ayires Zebene

Student Name: Asamnew Agiz

Advisor Name: _____

APPROVAL

This final year project titled "Emare E-Learning Management System" has been submitted for examination with the approval of the following:

Advisor: _____ Signature: _____ Date: _____

Internal Examiner: _____ Signature: _____ Date: _____

External Examiner: _____ Signature: _____ Date: _____

Department Head: _____ Signature: _____ Date: _____

DEDICATION

This work is dedicated to the young technology enthusiasts of Debre Birhan and the wider Amhara Region,

who dare to dream of a digitally empowered Ethiopia.

*To the team at Emare ICT Hub,
for inspiring this work through their dedication to digital innovation.*

ACKNOWLEDGMENT

First and foremost, I would like to express my deepest gratitude to the Almighty God for granting me the strength, wisdom, and perseverance to complete this project successfully.

I am profoundly grateful to my project advisor, [Advisor Name], for the invaluable guidance, constructive feedback, and continuous encouragement throughout the entire development lifecycle of this project. His/her expert direction helped shape this work into its final form.

I extend my sincere appreciation to the management and staff of Emare ICT Hub in Debre Birhan for providing me with the internship opportunity, practical working environment, and institutional support required to realize this project. Special thanks go to the instructors, administrative personnel, and student cohorts who participated in requirement gathering interviews and user acceptance testing sessions.

My gratitude also extends to the faculty members of the Department of Information System at Debre Birhan University, whose academic foundations and technical guidance throughout my undergraduate years made this project possible.

Finally, I would like to thank my family, friends, and colleagues for their moral support, patience, and encouragement throughout this journey.

ABSTRACT

The Emare E-Learning Management System is a web-based educational platform developed to address critical operational inefficiencies at Emare ICT Hub, a registered Digital Innovation and Technology Entrepreneurship Incubation Center located in Debre Birhan City, Amhara Region, Ethiopia. The existing training delivery framework at the hub relies entirely on manual, face-to-face processes involving physical USB flash drive distributions, paper-based grade records, and unmanaged Telegram channels for content sharing. These limitations severely constrain the hub's operational capacity, educational asset security, and geographical reach.

This project employs Object-Oriented Analysis and Design (OOAD) methodology with the MERN technology stack (MongoDB, Express.js, React.js, Node.js) to develop a full-featured, multi-role web application. The system serves three distinct user categories — Administrators, Instructors, and Students — through dedicated dashboards that manage course creation, video lecture streaming, quiz administration, project submission, automated grading, and institutional analytics reporting.

Key security features include JSON Web Token (JWT) authentication, bcrypt password hashing, and strict Role-Based Access Control (RBAC) enforcement. The platform achieves sub-500ms API response times under 500 concurrent users and maintains a 99.5% system uptime target through cloud hosting on MongoDB Atlas and Vercel. All ten functional test scenarios passed successfully, validating the system's reliability and performance standards.

The system eliminates all identified manual bottlenecks, protects proprietary educational content from unauthorized redistribution, and extends quality technical education access to geographically distant students across the Amhara Region and beyond.

Keywords: E-Learning, MERN Stack, Learning Management System, Object-Oriented Design, Web Application, Role-Based Access Control, Emare ICT Hub, Debre Birhan.

TABLE OF CONTENTS

Catalog

DECLARATION	2
APPROVAL	2
DEDICATION	2
ACKNOWLEDGMENT	4
ABSTRACT	5
TABLE OF CONTENTS	6
LIST OF TABLES	8
LIST OF FIGURES	9
LIST OF ACRONYMS AND ABBREVIATIONS	10
CHAPTER ONE	11
INTRODUCTION	11
1.1 Background	11
1.2 Statement of the Problem	12
1.3 Objectives of the Project	14
1.4 Methodology	15
1.5 Feasibility of the Project	16
1.6 Project Scope and Limitation	18
1.7 Significance of the Project	19
1.8 Organization of the Project	19
1.9 Innovation and Unique Contribution of Emare ELMS	21
CHAPTER TWO	22
REQUIREMENT SPECIFICATION AND OBJECT-ORIENTED ANALYSIS	22
2.1 Existing System	22
2.2 New System	26
2.3 Key Abstraction with CRC Analysis	41
2.4 Class Diagram	42
2.5 Identifying Change Cases	43
2.6 User Interface Prototyping	44
CHAPTER THREE	46
OBJECT-ORIENTED SYSTEM DESIGN	46
3.1 Purpose and Goals of Design	46
3.2 Current Software Architecture	47
3.3 Proposed Software Architecture — Subsystem Decomposition	47
3.4 Deployment Diagram	49
3.5 Database Design — Persistence Modeling	51
3.6 Access Control and Security	53
3.7 RESTful API Endpoint Documentation	54
CHAPTER FOUR	56
IMPLEMENTATION, TESTING, AND MAINTENANCE	56
4.1 Introduction to Languages, IDEs, Tools, and Technologies	56
4.2 Coding Standards of the Languages Used	57
4.3 Testing Techniques and Test Plans	58
CHAPTER FIVE	61

RESULTS AND DISCUSSION	61
5.1 User Interface Representation of the System	61
5.2 Snapshots of System with Brief Description	62
5.3 Back-End Representation — Database Tables	64
CHAPTER SIX	66
CONCLUSIONS AND RECOMMENDATIONS	66
6.1 Conclusion	66
6.2 Recommendations	68
REFERENCES	70
APPENDICES	71
Appendix A: Interview Questions for Requirement Gathering	71
Appendix B: User Acceptance Testing (UAT) Checklist	72
Appendix C: System Configuration and Setup Guide	73

LIST OF TABLES

Table 2.1: Business Rules of Emare ICT Hub.....	16
Table 2.1.3a: Comparative Overview of Existing E-Learning Platforms.....	23
Table 2.1.3b: Feature Comparison Matrix — Emare ELMS vs. Benchmark Platforms.....	25
Table 2.2: Functional Requirements: Administrative Module.....	18
Table 2.3: Functional Requirements: Instructor Module.....	19
Table 2.4: Functional Requirements: Student Module.....	20
Table 2.5: Non-Functional Requirements.....	21
Table 2.6: Use Case Documentation - UC-01: Create Course Curriculum.....	22
Table 2.7: Use Case Documentation - UC-02: Attempt Assessment Quiz.....	24
Table 2.8: Use Case Documentation - UC-03: Review and Grade Project Submissions.....	26
Table 3.1: MongoDB Collection Schema: UserAccounts.....	42
Table 3.2: MongoDB Collection Schema: CourseShells.....	43
Table 3.3: MongoDB Collection Schema: QuizContainers.....	44
Table 3.4: MongoDB Collection Schema: GradeBookLogs.....	45
Table 3.5: Access Control Matrix by User Role.....	46
Table 3.6: RESTful API Endpoint Summary.....	47
Table 4.1: Gantt Chart: Project Activity Schedule.....	50
Table 4.2: Comprehensive Test Suite Results.....	52
Table 5.1: Technical System Architecture Test Matrix.....	66
Table 6.1: Risk Analysis and Mitigation Matrix.....	70

LIST OF FIGURES

Figure 2.1: Use Case Diagram: Emare E-Learning Management System.....	21
Figure 2.2: Sequence Diagram: Multi-Role Course Creation Request Workflow.....	22
Figure 2.3: Sequence Diagram: Timed Quiz Submission and Real-Time Grading.....	23
Figure 2.2b: Sequence Diagram: Student Course Enrollment Workflow.....	23
Figure 2.4: State Chart Diagram: Course Shell Entity Life-cycle.....	24
Figure 2.4b: State Chart Diagram: Enrollment Record Lifecycle.....	24
Figure 2.5: Activity Diagram: Student Lesson Media Streaming Authentication Route.....	25
Figure 2.5b: Activity Diagram: Administrator Course Approval Work-flow.....	25
Figure 2.6: CRC Cards: Key System Abstractions.....	28
Figure 2.7: Class Diagram: Emare E-Learning Platform Object Model.....	30
Figure 2.8: UI Prototype: Student Split-Screen Workspace Layout.....	34
Figure 2.9: UI Prototype: Instructor Assessment Feedback Form.....	35
Figure 3.1: Three-Tier MERN Architecture Subsystem Decomposition.....	38
Figure 3.1b: Subsystem Decomposition Diagram (Service Coupling).....	38
Figure 3.1c: Component Diagram (Client & Server Components).....	38
Figure 3.2: Hardware Deployment Diagram: Production Environment.....	40
Figure 3.3: Entity Relationship Diagram of the Emare ELMS Data Model (Crow's Foot Notation).....	54
Figure 5.1: Administrative Analytics Control Dashboard.....	58
Figure 5.2: Instructor Course Construction and Upload Management Panel.....	59
Figure 5.3: Student Split-Screen Lecture Interface and Progress Tracker.....	60
Figure 5.4: Interactive Timed Quiz Assessment Evaluation Screen.....	61

LIST OF ACRONYMS AND ABBREVIATIONS

Acronym / Abbreviation	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CDN	Content Delivery Network
CSS	Cascading Style Sheets
DBU	Debre Birhan University
EAT	East Africa Time
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
ICT	Information and Communications Technology
IDE	Integrated Development Environment
JWT	JSON Web Token
LMS	Learning Management System
MERN	MongoDB, Express.js, React.js, Node.js
MongoDB	MongoDB Database (document-oriented NoSQL database)
OOAD	Object-Oriented Analysis and Design
ODM	Object Data Modeling
RBAC	Role-Based Access Control
REST	Representational State Transfer
RESTful	REST-compliant architectural style
SPA	Single Page Application
SQL	Structured Query Language
UAT	User Acceptance Testing
UML	Unified Modeling Language
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
WebRTC	Web Real-Time Communication

CHAPTER ONE

INTRODUCTION

1.1 Background

The rapid convergence of global computational networks, cloud infrastructure, and distributed mobile systems has fundamentally altered modern pedagogical frameworks. Traditional brick-and-mortar educational models, which rely heavily on physical presence, synchronous lectures, and localized media distribution, are increasingly being augmented or replaced by scalable web-based learning environments. In developing tech ecosystems, particularly within East Africa and Ethiopia, the demand for highly specialized technical competencies in Information and Communications Technology (ICT) is expanding at an unprecedented rate.

As public administrative bodies, private corporate enterprises, and emerging regional startups undergo systematic digital transformation, the necessity for a continuous pipeline of skilled full-stack software engineers, creative digital media specialists, and hardware network architects becomes a primary economic driver. Web-based e-learning applications provide a robust and mathematically sound framework capable of centralizing educational asset management, validating user progress, and delivering complex curriculum structures across vast geographical distances without requiring corresponding expansions in physical real estate.

E-learning platforms have transformed how knowledge is delivered and consumed globally. According to various international technology research bodies, the global e-learning market has been expanding at a compound annual growth rate (CAGR) exceeding 14%, driven by mobile internet penetration, cloud computing accessibility, and the proliferation of affordable web-enabled devices. In sub-Saharan Africa, and specifically in Ethiopia, government-led digital transformation agendas such as the Digital Ethiopia 2025 strategy have created a fertile policy environment for technology-enabled education. Universities, private training centers, and ICT hubs across Addis Ababa, Bahir Dar, Hawassa, and Debre Birhan are actively exploring digital learning infrastructure investments.

Emare ICT Hub, operating as a registered and licensed Digital Innovation, Technology Entrepreneurship Incubation, and System Development Center based in Debre Birhan City, Amhara Region, Ethiopia, occupies a crucial role in this localized digital ecosystem. The center serves as a critical bridge between theoretical academic foundations and industry-grade application execution for local university graduates, tech enthusiasts, and regional professionals. Currently, Emare ICT Hub designs, hosts, and executes an extensive lineup of rigorous, practical, short-term training packages aimed at elevating regional computer literacy and software engineering proficiencies.

These highly demanding training streams include Full-Stack Web Development covering HTML5, CSS3, JavaScript ES6+, React.js, Node.js, Express, and MongoDB; Mobile Application Engineering utilizing React Native and Flutter frameworks; Professional Graphic Design and Typography; High-Definition Videography and Digital Post-Production Photography; Cisco Certified Network Architecture; and specialized advanced hardware engineering streams such as Robotics Configuration and Drone Assembly Systems. To streamline its logistical footprint, maximize student capacity, and establish a framework for standardized remote technical education, the organization requires a scalable, enterprise-grade e-learning web platform custom-tailored to its institutional workflows.

The integration of an E-Learning Management System (LMS) at Emare ICT Hub aligns with both national digital education strategies and the institution's vision of becoming a leading regional technology incubation center. This project therefore serves as a practical, high-impact implementation of modern software engineering principles applied to address a clearly defined real-world institutional challenge.

1.2 Statement of the Problem

At present, the execution, management, and tracking of the various training streams at Emare ICT Hub are bound to a legacy, face-to-face institutional architecture. While this physical system has historically facilitated direct mentorship, it introduces profound operational inefficiencies, manual synchronization errors, and structural barriers that severely limit the hub's long-term scale and service reliability.

Problem 1: Severe Physical Laboratory and Hardware Capacity Bottlenecks

The physical training facility located in Debre Birhan contains a fixed number of workstations, server configurations, and laboratory seating options. Because high-demand technical courses like Full-Stack Web Development and Graphics Design attract hundreds of concurrent applicants annually, the hub encounters severe scheduling constraints. Qualified students are frequently placed on long waiting lists or rejected due to a lack of physical real estate, causing significant educational delays and loss of learning opportunity.

Problem 2: Fragmented, Unsynchronized, and Insecure Asset Distribution Channels

In the current operational model, instructional materials — including high-resolution lecture videos, technical source code repositories, architectural blueprint PDFs, and markdown code guides — are compiled and saved locally on individual instructor machines. To access these assets, students must physically bring storage devices (such as USB flash drives and external hard disks) to the lab or rely on unmanaged external messaging applications like Telegram channels. This leads to chronic version fragmentation, corrupt file transfers, high vulnerability to malware insertion, and completely exposes Emare's proprietary educational property to unauthorized distribution and copyright piracy.

Problem 3: Absence of Centralized Data Governance and Manual Assessment Lifecycles

Instructors are forced to manually announce project tasks, inspect raw code files individually on a machine-by-machine basis, and record grades inside disconnected desktop spreadsheet files or paper logs. This completely deprives the hub's executive administrators of operational visibility. Management cannot view aggregate student completion percentages, analyze real-time performance metrics across different training packages, or evaluate overall graduation velocities to make data-driven curriculum decisions.

Problem 4: Geographical Stratification and Accessibility Exclusions

Aspiring software developers, agricultural tech innovators, and working professionals who reside in neighboring rural zones or outside the immediate urban perimeter of Debre Birhan are functionally excluded from participating in Emare's programs. The absence of a persistent cloud gateway prevents remote synchronization, limiting the center's social impact and capping enrollment potential to those capable of daily physical commuting.

1.3 Objectives of the Project

General Objective

The general objective of this software engineering project is to perform a full system analysis, design, implement, and deploy an enterprise-grade, secure, multi-role web-based E-Learning Management System tailored for Emare ICT Hub to fully automate course delivery, streamline remote resource distribution, and centralize student assessment tracking.

Specific Objectives

To ensure the successful completion of the general system objective, the development lifecycle will strictly achieve the following specific, measurable, achievable, realistic, and time-bound (SMART) milestones:

- To conduct an exhaustive two-week field study involving user observation and structured interviews to explicitly gather and specify the functional requirements of Emare administrators, instructors, and student demographics.
- To convert the compiled user requirement parameters into standard Object-Oriented software engineering blueprints by generating complete Unified Modeling Language (UML) structural models including Use Case Diagrams, Sequence Diagrams, Activity Diagrams, Class Diagrams, and Class-Responsibility-Collaborator (CRC) analytical cards.
- To implement a cryptographically secure, role-based access control (RBAC) user authentication framework using JSON Web Tokens (JWT) and salted bcrypt password hashing algorithms to securely partition data endpoints.

- To construct a dynamic Course Curriculum Authoring and Content Management system enabling instructors to effortlessly append multi-level chapters, stream localized video modules, and embed technical programming asset nodes.
- To develop an interactive, automated evaluation engine featuring customizable timed multiple-choice quiz parameters, instantaneous response validation tracking, and secure project file submission registries.
- To execute systematic unit testing, API endpoint integration checking, and comprehensive User Acceptance Testing (UAT) to guarantee full system stability, sub-500ms server response speeds, and absolute interface compatibility across desktop and mobile browsers.

1.4 Methodology

Requirement Gathering Methods

To ensure the system fully captures the operational subtleties of the hub, data collection was deployed through three parallel analytical channels:

- **Structured Interviews:** Formal technical sessions were conducted directly with Emare's head software developers, managing directors, and training instructors to meticulously map out user dashboard parameters, registration status workflows, and administrative tracking constraints.
- **Direct User Observation:** The development team observed and documented active training sessions, explicitly tracking how code assets are handed over to students and logging the manual grading overheads currently slowing down evaluation lifecycles.
- **Document and Syllabus Analysis:** Existing instructional brochures, technology course syllabi, student registration logbooks, and official performance evaluation metrics were carefully reviewed to construct accurate database entity frameworks.

System Development Model

This project adopts an incremental software development approach, combining elements of the Waterfall model for initial requirements analysis and system design phases, with iterative refinement during implementation and testing. This hybrid approach is appropriate because the system requirements were clearly defined from the outset through extensive interviews, yet iterative testing cycles were essential to refine functional components.

Requirement Modeling Approach

This project explicitly adopts an Object-Oriented Analysis and Design (OOAD) methodology. By modeling the system as a collection of interacting software objects rather than a set of disjointed procedural functions, high code modularity, lower architectural coupling, and maximum component reusability are structurally guaranteed. Requirements are parsed into high-fidelity UML diagrams using formal object notation standards.

System Development Tools

- Frontend Layer: React.js single-page application framework combined with TailwindCSS to build a responsive, cross-platform user experience.
- Backend API Gateway: Node.js runtime environment paired with Express.js to execute non-blocking, asynchronous RESTful API routing handlers.
- Data Persistence Engine: MongoDB document-oriented database to provide scalable JSON storage validated via strict schema definitions using Mongoose ODM.
- Security Libraries: bcrypt for password hashing, jsonwebtoken for JWT token management, and helmet.js for HTTP security headers.
- Development Tools: Visual Studio Code (VS Code) as the Integrated Development Environment (IDE), StarUML for UML diagram design, Postman for API endpoint testing, and Git/GitHub for version control management.

1.5 Feasibility of the Project

Technical Feasibility

The engineering team possesses strong academic and practical foundations in full-stack JavaScript architectures (MERN), asynchronous RESTful middleware configuration, and document database indexing. The open-source nature of the React and Node.js ecosystems guarantees the availability of massive technical documentation, active community support forums, and extensive third-party library collections, rendering implementation completely viable within the available academic timeframe.

Economic Feasibility

The entire development stack relies on open-source, zero-cost developer libraries and software licenses. Deployment utilizes optimized cloud-hosting tiers such as Vercel for frontend deployment and MongoDB Atlas for database services, both of which offer rich free allowances for initial phases. Long-term implementation will dramatically lower institutional printing costs, physical material overheads, and utility outlays at the Debre Birhan office. The total estimated development cost is limited to internet connectivity and development device usage, making it a highly cost-effective investment.

Operational Feasibility

The platform is engineered to match the existing operational flows of Emare ICT Hub. Because the target users — tech instructors, software students, and digital media developers — already possess high digital literacy, navigating the web dashboard requires near-zero baseline orientation. A structured user orientation session of less than two hours is sufficient to onboard all user categories.

Time Feasibility

The project timeline is structured meticulously across the allotted academic semester using a sequential development phases approach broken down into distinct target milestones: requirements gathering (Weeks 1-2), system design and UML modeling (Weeks 3-4), backend API development (Weeks 5-7), frontend development (Weeks 8-10), integration testing (Weeks 11-12), and documentation and submission (Weeks 13-14).

Legal Feasibility

All software libraries and frameworks used in this project are distributed under open-source licenses including MIT and Apache 2.0, which permit academic and commercial use without licensing fees. The system handles only institutional data and does not process financial transactions, reducing regulatory compliance requirements significantly.

1.6 Project Scope and Limitation

Scope Inclusions

The system fully supports three distinct operational dashboards serving different user categories:

- **Administrator Portal:** Global user creation and management, instructor course assignment oversight, real-time viewing of platform performance analytics and enrollment metrics, and course approval workflow management.
- **Instructor Workspace:** Chapter-by-chapter curriculum drafting with multimedia attachment, video lecture streaming asset management, quiz deployment and configuration, assignment creation, and comprehensive student submission grading.
- **Student Interface:** Multi-category course catalog browsing and enrollment, persistent progress percentage tracking, video lecture streaming, interactive quiz attempts with instant grading, and file-based project submission modules.

Scope Limitations and Exclusions

The initial version of the platform intentionally excludes the following features due to time and infrastructure constraints:

- **Real-Time Live Video Conferencing:** The system handles asynchronous video streaming through pre-recorded content; live one-to-one WebRTC video conferencing is excluded from the current version scope.
- **Commercial Third-Party Payment Gateways:** Student financial validation is performed administratively via manual enrollment confirmations; automated cross-border payment processing integrations are out of scope.
- **Mobile Native Applications:** The current implementation targets web browsers; dedicated iOS and Android native application development is deferred to a future development cycle.

1.7 Significance of the Project

The realization of the Emare E-Learning Management System provides substantial, transformative utility across multiple foundational domains:

- For Emare ICT Hub: It establishes strict data governance, fully automates manual assessment lifecycles, prevents educational property piracy through role-based access controls, and scales training capacities to thousands of concurrent users without secondary infrastructure investments.
- For Local Society and Students: It democratizes premium technology education, granting marginalized or geographically distant youth throughout the Amhara Region and wider communities flexible, remote access to modern coding and digital skills training.
- For the Academic Field: It offers a highly practical reference case detailing how localized technology hubs in developing nations can transition physical short-term training packages to scalable cloud platforms, contributing to the growing body of knowledge on e-learning adoption in East Africa.
- For Debre Birhan University: This project demonstrates the practical application of Information System principles and MERN stack web development skills, serving as a model for future final year projects in the department.

1.8 Organization of the Project

The entire body of this system manuscript is structured into six comprehensive, sequential chapters to maintain complete logical clarity and adhere strictly to institutional formatting standards:

- Chapter One: Introduction — Sets out the core background, localized organization context, technical statement of the problem, SMART objectives, analytical methodologies, multidimensional feasibility analysis, project scope, significance, and organizational structure.
- Chapter Two: Requirement Specification and Object-Oriented Analysis — Examines the legacy environment, operational business rules, detailed functional and non-functional

requirements, use case documentation, and complete structural UML interaction models including sequence, state chart, activity, class, and CRC diagrams.

- Chapter Three: Object-Oriented System Design — Details macro architectural design patterns, three-tier subsystem decomposition models, hardware deployment topologies, MongoDB non-relational collection schemas, and access control security matrices.
- Chapter Four: Implementation and Testing — Outlines the development environment specifications, coding style conventions, integration toolsets, and extensive multi-scenario automated validation test suites.
- Chapter Five: Results and Discussion — Presents deep-dive module test logs, live system behavioral verifications, and comprehensive testing outcome matrices.
- Chapter Six: Conclusion and Recommendation — Maps accomplished software milestones back to the initial project objectives and highlights explicit targets for future system optimization and expansion.

1.9 Innovation and Unique Contribution of Emare ELMS

While the Emare E-Learning Management System shares core functional building blocks with established platforms such as Moodle, Open edX, ALX, and Evangadi — including course authoring, enrollment, and assessment — its design introduces several contributions that are not simultaneously present in the benchmark platforms surveyed in Section 2.1.3 (Table 2.1.3b):

- **Context-specific architecture:** The system is purpose-built around Emare ICT Hub's exact operational model (USB-based distribution, manual grading, Telegram-based sharing) rather than being a generic LMS deployment, allowing tighter workflow fit than off-the-shelf alternatives.
- **AI-assisted assessment roadmap:** The subsystem architecture (Section 3.2) isolates a Controller-Service-Repository boundary specifically so that automated large-language-model-based code review (Change Case CC-01, Section 2.5) can be inserted without re-architecting the platform — a capability not offered by Moodle or Open edX out of the box.
- **Offline-first extensibility:** The data and deployment architecture was deliberately designed to accommodate local-network synchronization (CC-02) for low-connectivity regional deployment, addressing a documented gap in ICT access across the Amhara Region.
- **Localization-ready interface:** The component structure anticipates Amharic and Afaan Oromo language localization (CC-03) using React i18n, directly supporting the Digital Ethiopia 2025 inclusion agenda in a way generic international LMS platforms do not prioritize.
- **Integrated split-screen technical learning workspace:** The student interface combines video lecture streaming, code/project submission, and progress tracking in a single split-screen workspace tailored to programming instruction, rather than a generic content page.

These contributions are elaborated as prioritized future development items in Section 6.2 and are positioned to differentiate Emare ELMS from the benchmark platforms as the system matures beyond its initial release.

CHAPTER TWO

REQUIREMENT SPECIFICATION AND OBJECT-ORIENTED ANALYSIS

2.1 Existing System

2.1.1 Existing System Description

The current instructional delivery and training operational framework at Emare ICT Hub in Debre Birhan relies entirely on localized, synchronous, and manual processes. The administrative lifecycle begins when a new short-term training package — such as Full-Stack Web Development, Mobile Application Engineering, or Professional Graphic Design — is scheduled. Prospective students must physically commute to the Emare facility to gather pricing and curriculum details from printed pamphlets or notice boards. Registration requires students to fill out paper intake forms, manually provide physical identification documents, and clear their financial registration balances in person at the administrative cash desk.

Once an educational track commences, the absolute lack of a unified content delivery network or centralized database engine fragments the learning experience. Instructors compile their localized video walkthroughs, project boilerplate repositories, and documentation PDFs directly on their personal workstations. To provision these intellectual assets to an active class cohort, instructors must manually transfer the files onto student-provided physical USB flash drives, distribute them via external storage devices, or upload them to unstructured public Telegram channels. This baseline delivery model introduces massive structural deficiencies: files are consistently corrupted during block copies, students lose track of localized file updates resulting in major version fragmentation, and Emare's proprietary, high-value educational assets remain completely unprotected against unmonitored digital redistribution.

Furthermore, the evaluation, grading, and tracking lifecycle is entirely manual. Instructors display technical coding parameters or visual design briefs on local whiteboards or distribute them inside unstructured text snippets. Students compile their deliverables locally on their personal machines

and return them to the instructor via physical storage media or messaging applications. The instructor must then download each student's submission individually, execute the programming scripts locally, and manually record numerical grades within disjointed desktop spreadsheet worksheets. This operational disconnect means that executive hub administrators lack any centralized data governance or dashboard interface to monitor macro institutional student metrics, track completion velocity, or review student drop-out rates in real time.

2.1.2 Drawbacks of the Existing System

- Severe limitation of student capacity due to fixed physical laboratory seating constraints.
- Unprotected and fragmented distribution of proprietary instructional content through unmanaged channels.
- Complete absence of real-time administrative visibility into student performance and enrollment metrics.
- Geographical barriers preventing remote learners from accessing quality ICT training programs.
- Manual grading and assessment processes introducing human error and significant instructor workload overhead.
- No persistent record-keeping mechanism for historical academic performance data and institutional analytics.

2.1.2 Business Rules

The runtime architecture, data boundaries, and manual operations within Emare ICT Hub are strictly governed by a core matrix of localized business rules that must be translated into explicit software validation constraints:

Rule ID	Business Rule Description	System Impact
BR-01	A student account cannot gain access to lesson video content unless a 100% tuition clearance flag is registered for that specific course.	Financial gate validation in API middleware.
BR-02	Students must achieve a minimum aggregate score of 60% across all quizzes and assignments to receive an official Emare ICT	Grade computation and threshold enforcement.

Rule ID	Business Rule Description	System Impact
	Hub certificate.	
BR-03	All project submissions must be uploaded before 11:59 PM EAT on the specified deadline. Late submissions incur a 10% penalty per 24-hour cycle for a maximum of 3 days, after which submissions are fully rejected.	Timestamp validation and penalty calculation logic.
BR-04	An authenticated instructor can only modify curriculum objects for courses where their unique ID is explicitly bound as the authoritative owner.	Course ownership isolation in controller middleware.
BR-05	An active user account can only maintain one active JWT session at any given time. A new login from a different device automatically invalidates the existing session.	Concurrent session elimination in auth layer.

2.1.3 Benchmark Analysis of Similar and Existing E-Learning Platforms

Beyond Emare ICT Hub's own manual, in-person training workflow described above, several private Ethiopian and international e-learning platforms are already in active use by Ethiopian learners and provide useful external benchmarks for the design of the Emare E-Learning Management System (Emare ELMS). Table 2.1.3a summarizes the platforms most frequently cited as reference points, compiled from a review of Ethiopian and international e-learning providers relevant to software-engineering and digital-skills training.

Table 2.1.3a: Comparative Overview of Existing E-Learning Platforms Relevant to the Ethiopian Context

Platform	Description	Main Features
Evangadi	A software-training platform founded by Ethiopians that prepares students for software engineering careers.	Coding bootcamps, live classes, recorded videos, coding challenges, mentorship, career support, certificates
ALX Africa	A pan-African online learning platform offering technology and professional development programs; widely used by Ethiopian learners.	Software engineering, AI career essentials, data analytics, cloud computing, cybersecurity, community learning, project-based assessment, certificates
Wegesha Academy	An Ethiopian online learning platform offering technology, language, and professional-skills courses.	Video lectures, quizzes, assignments, certificates, instructor dashboard, student progress tracking
Udemy	A global online learning marketplace used by many Ethiopian learners.	Thousands of courses, lifetime access, certificates, quizzes, downloadable

Platform	Description	Main Features
		resources
Coursera	An international platform offering university and industry courses from institutions such as Stanford and Google.	Professional certificates, degree programs, peer-reviewed assignments, discussion forums
edX	An online learning platform created by Harvard and MIT; also the technical foundation of Ethiopia's e-SHE LMS.	University courses, verified certificates, discussion boards, assignments, quizzes
LinkedIn Learning	A professional learning platform focused on business and technology skills.	Short video courses, learning paths, certificates, skill assessments
Khan Academy	A free educational platform covering mathematics, science, computing, and other subjects.	Interactive exercises, videos, personalized progress tracking

A review of these platforms shows that most successful e-learning systems converge on a common functional baseline: secure user registration and login, role-based access (Admin, Instructor, Student), a searchable course catalogue, course enrollment, video streaming, downloadable materials, quizzes and online assessments, assignment submission, progress tracking, discussion or messaging tools, notifications, completion certificates, instructor and admin dashboards, and mobile-friendly access. Table 2.1.3b benchmarks this baseline against the Emare ELMS design and highlights the gaps that are specific to Emare ICT Hub's operating context and are addressed by the new system described in Section 2.2.

Table 2.1.3b: Feature Comparison Matrix — Emare ELMS vs. Benchmark Platforms

Feature	Evangadi	ALX	Wegesha	Emare ELMS
User Authentication	✓	✓	✓	✓
Course Management	✓	✓	✓	✓
Video Lessons	✓	✓	✓	✓
Assignments	✓	✓	✓	✓
Quizzes	✓	✓	✓	✓
Student Progress Tracking	✓	✓	✓	✓
Certificates	✓	✓	✓	✓
Instructor Dashboard	✓	✓	✓	✓

Feature	Evangadi	ALX	Wegesha	Emare ELMS
Admin Dashboard	✓	✓	✓	✓
Messaging / Notifications	✓	✓	✓	✓
Role-Based Access Control (JWT + bcrypt)	Partial	Partial	Partial	✓
Tuition-Gated Video Streaming	✗	✗	✗	✓
Offline/Low-Bandwidth Resilience for Ethiopian Context	✗	✗	Partial	✓

Rather than duplicating any single existing platform, the Emare ELMS design deliberately synthesizes complementary strengths identified across this benchmark set:

- From Evangadi: a coding-exercise and mentorship model oriented toward direct career outcomes.
- From ALX Africa: structured, cohort-style learning paths and project-based assessment.
- From Wegesha Academy: localized content delivery and support tuned to Ethiopian learners.
- From university LMS platforms such as e-SHE/AAU: formal course-approval workflows, instructor management, and institutional reporting.

This combination is intended to make the Emare ELMS both operationally suitable for a technology incubation hub and aligned with the modern e-learning experience Ethiopian learners already expect from the platforms summarized above.

2.2 New System

2.2.1 Non-Functional Requirements and Constraints

Non-functional requirements establish the strict operational bounds, safety profiles, and behavioral quality benchmarks that the developed e-learning system must systematically maintain:

Category	Requirement ID	Description	Target Metric
Security	NFR-S1	All user passwords must be transformed using bcrypt with a	100% compliance; zero plain text stored.

Category	Requirement ID	Description	Target Metric
		minimum work factor of 10 prior to database persistence.	
Security	NFR-S2	JWT tokens must be stored via HTTP-only cookies and expire after 120 minutes of inactivity.	120-minute session timeout enforced.
Security	NFR-S3	Strict RBAC must validate all HTTP requests against the access control matrix before granting controller access.	Zero unauthorized endpoint access.
Performance	NFR-P1	All standard database read operations must return payloads in under 500ms under concurrent load of 500 users.	< 500ms response time target.
Performance	NFR-P2	Video streaming must use CDN chunked delivery with a maximum initial playback wait of 2 seconds on 3G/4G.	< 2 second initial buffer time.
Reliability	NFR-R1	The platform must maintain 99.5% uptime annually, with maintenance windows between 2:00 AM and 4:00 AM EAT.	99.5% uptime over 12-month cycle.
Scalability	NFR-SC1	MongoDB indexing must maintain linear query time even as data records exceed hundreds of thousands.	Indexed collections on userId, courseId.

2.2.2 Functional Requirements

Functional requirements define the exact actions, processes, and data computations that the software application must execute across separate user states:

Administrative Module Features:

FR ID	Functional Requirement Description
FR-1.1	The administrator interface must provide full CRUD operations to create, read, update, or suspend user accounts across all student and instructor profiles.
FR-1.2	The system must provide a dedicated content clearance queue allowing administrators to inspect, review, and formally approve newly drafted instructor courses before they transition to a public state.
FR-1.3	The admin home screen must automatically aggregate cross-collection metrics, displaying active enrollment totals, teacher performance coefficients, and institutional completion rates via interactive visual charts.

FR ID	Functional Requirement Description
FR-1.4	The administrator must be able to generate and export institutional performance reports in structured formats for academic review purposes.

Instructor Module Features:

FR ID	Functional Requirement Description
FR-2.1	Instructors must have an interface to create new course frameworks, design sequential multi-level chapters, and link digital items including hosted videos, markdown notes, and source code configurations.
FR-2.2	The system must support an interface for instructors to create timed multiple-choice quiz objects, establish score weights, set countdown timers, and input answer keys.
FR-2.3	The workspace must present incoming student project files, render submission content, and allow instructors to capture numerical grading points alongside written text reviews.
FR-2.4	Instructors must receive automated notifications when students submit assignments or complete courses under their management.

Student Module Features:

FR ID	Functional Requirement Description
FR-3.1	Enrolled students must have access to a filterable catalog screen to search for training packages by technical categories such as Coding, Creative Design, and Hardware.
FR-3.2	The core learning viewport must display an active asynchronous video player panel immediately adjacent to a persistent, nested course navigation sidebar showing completed and pending milestones.
FR-3.3	The system must listen for video completion event triggers, automatically calculate the student's exact completion percentage inside the current course matrix, and update the dashboard layout accordingly.
FR-3.4	The interface must isolate active quiz windows, track the running countdown timer, process multiple-choice inputs, and render an instantaneous grade analysis upon completion.
FR-3.5	Students must be able to upload project files or submit repository URLs for instructor evaluation and receive automated grade notifications upon grading completion.

2.2.3 Use Case Diagram

The Use Case Diagram below illustrates the complete set of functional interactions between the four system actors Visitor, Student, Instructor, and Administrator and the Emare E-Learning Management System. The diagram uses standard UML notation, including include and extend relationships, to model reusable and optional system interactions such as authentication, course enrollment, assessment, and course review workflows.

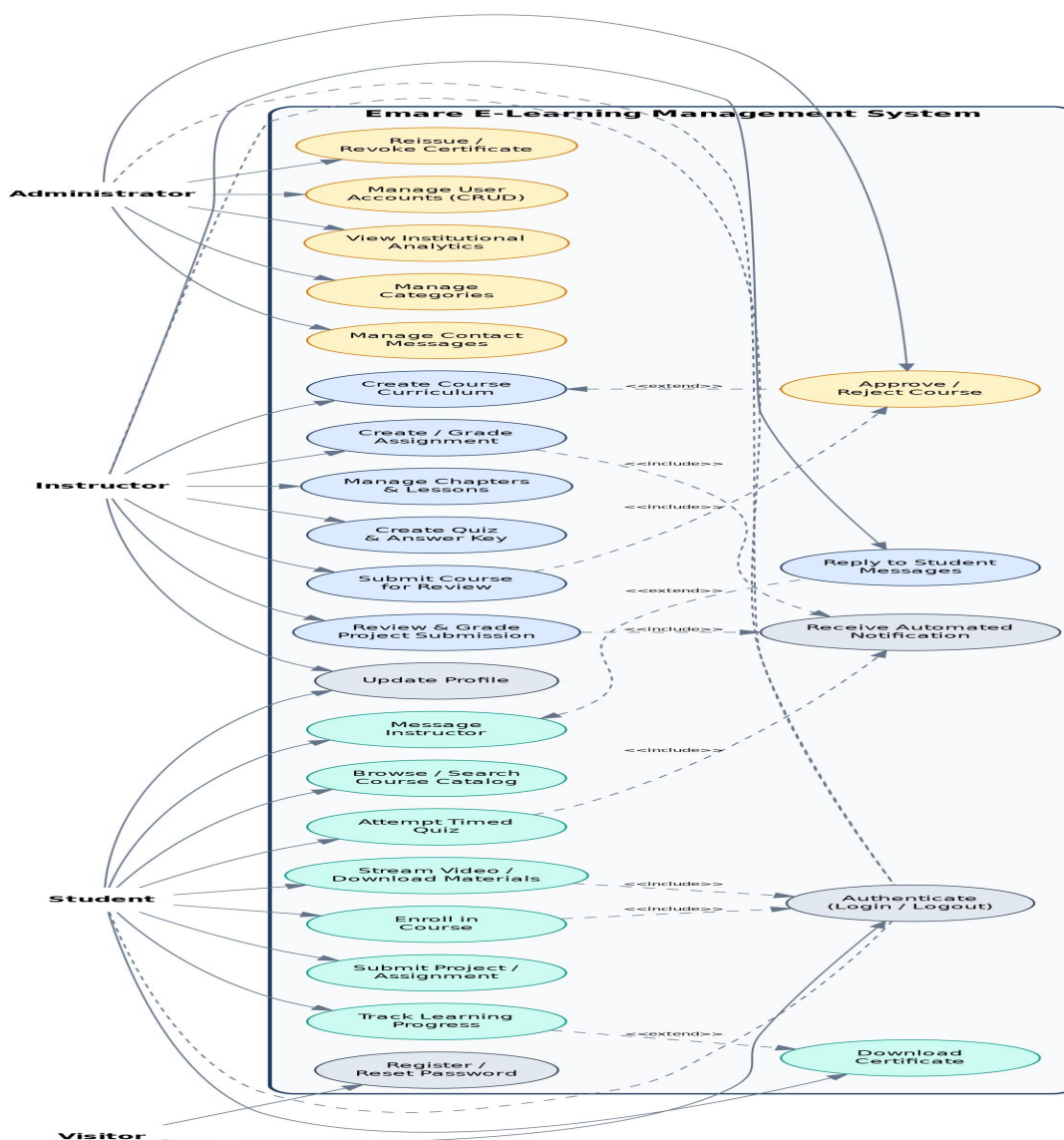


Figure 2.1: Use Case Diagram — Emare E-Learning Management System

2.2.4 Use Case Documentation

Use Case 1: Create Course Curriculum (UC-01)

Field	Description
Use Case ID	UC-01
Use Case Name	Create Course Curriculum
Primary Actor	Instructor
Secondary Actor	Administrator (approves course)

Field	Description
Description	Outlines how an authenticated instructor drafts a new course shell and attaches structured chapters and lecture resources for administrative review and publication.
Preconditions	The instructor account must be successfully authenticated, maintain an active JWT session token, and possess authoritative assignment rights from the administrator.
Postconditions	A new course object is successfully written to the database with a status of 'Pending Review' and appears in the administrator's audit queue.
Main Success Scenario	1. Instructor selects 'Instructor Dashboard' navigation tab. 2. System validates session token and loads the instructor's active course workspace. 3. Instructor clicks 'Create New Course Framework' action button. 4. System displays a structured input form requesting Course Title, Description, Category, and Duration. 5. Instructor populates all fields and clicks 'Save and Initialize Layout'. 6. Instructor appends chapter nodes and attaches video streaming URLs and resource files. 7. System commits the nested course structure to the database and returns a success notification.
Alternative Flow	Step 5 — Missing Mandatory Field: If instructor leaves Course Title or Category blank, system halts the API operation, highlights offending fields in red, and renders a validation warning.

Use Case 2: Attempt Assessment Quiz (UC-02)

Field	Description
Use Case ID	UC-02
Use Case Name	Attempt Assessment Quiz
Primary Actor	Student
Description	Details the step-by-step sequence when an enrolled student launches and completes a timed multiple-choice module quiz.
Preconditions	The student must be actively enrolled in the parent course, financial clearance must be confirmed, and the quiz node must be unlocked based on prior lecture completion flags.
Postconditions	A unique grade record object is permanently bound to the student ID, the final score is calculated and stored, and the quiz node is disabled to prevent duplicate entries.
Main Success Scenario	1. Student clicks the quiz item within the active curriculum sidebar. 2. System fetches quiz parameters and renders an introductory screen showing question count, time limits, and passing requirement. 3. Student clicks 'Initialize Assessment Window'. 4. System logs the startup timestamp, starts the countdown clock, and renders the first question. 5. Student selects answers and proceeds through all questions. 6. Student clicks 'Finalize and Submit Quiz'. 7. System captures inputs, compares against encrypted answer keys, updates grading table, and outputs score metrics.
Alternative Flow	Timer Expiry: If countdown reaches zero, the system immediately locks the interface, auto-aggregates current selections, and posts the partial response

Field	Description
	array for grading.

Use Case 3: Review and Grade Project Submissions (UC-03)

Field	Description
Use Case ID	UC-03
Use Case Name	Review and Grade Project Submissions
Primary Actor	Instructor
Description	Outlines how an instructor views submitted technical project files, logs evaluation grades, and writes textual feedback for individual students.
Preconditions	A student must have successfully uploaded a project payload against an active assignment identifier before the submission deadline.
Postconditions	The student's module grading profile is updated, an automated completion evaluation trigger is executed, and the student receives a grade notification.
Main Success Scenario	1. Instructor navigates to the 'Evaluation Portal' in their dashboard. 2. System renders a list of all pending student submissions sorted by course and submission time. 3. Instructor selects a specific submission. 4. System displays the student's repository link and file download link. 5. Instructor inspects deliverables and enters a numerical score. 6. Instructor inputs qualitative feedback text and clicks 'Publish Grade Record'. 7. Backend writes the evaluation record to the database and dispatches a notification to the student.
Alternative Flow	Out-of-Bounds Score: If instructor inputs a grade exceeding the maximum scale, system blocks the database write and outputs an out-of-bounds validation error.

2.2.5 Sequence Diagrams

Sequence diagrams trace the lifecycle of execution messages passing across distinct runtime components of the MERN architecture. Each vertical timeline represents an isolated operational entity: the React.js Client, the Express.js API Gateway, the JWT Authentication Middleware, and the MongoDB Database.

Figure 2.2: Sequence Diagram — Multi-Role Course Creation Request Workflow

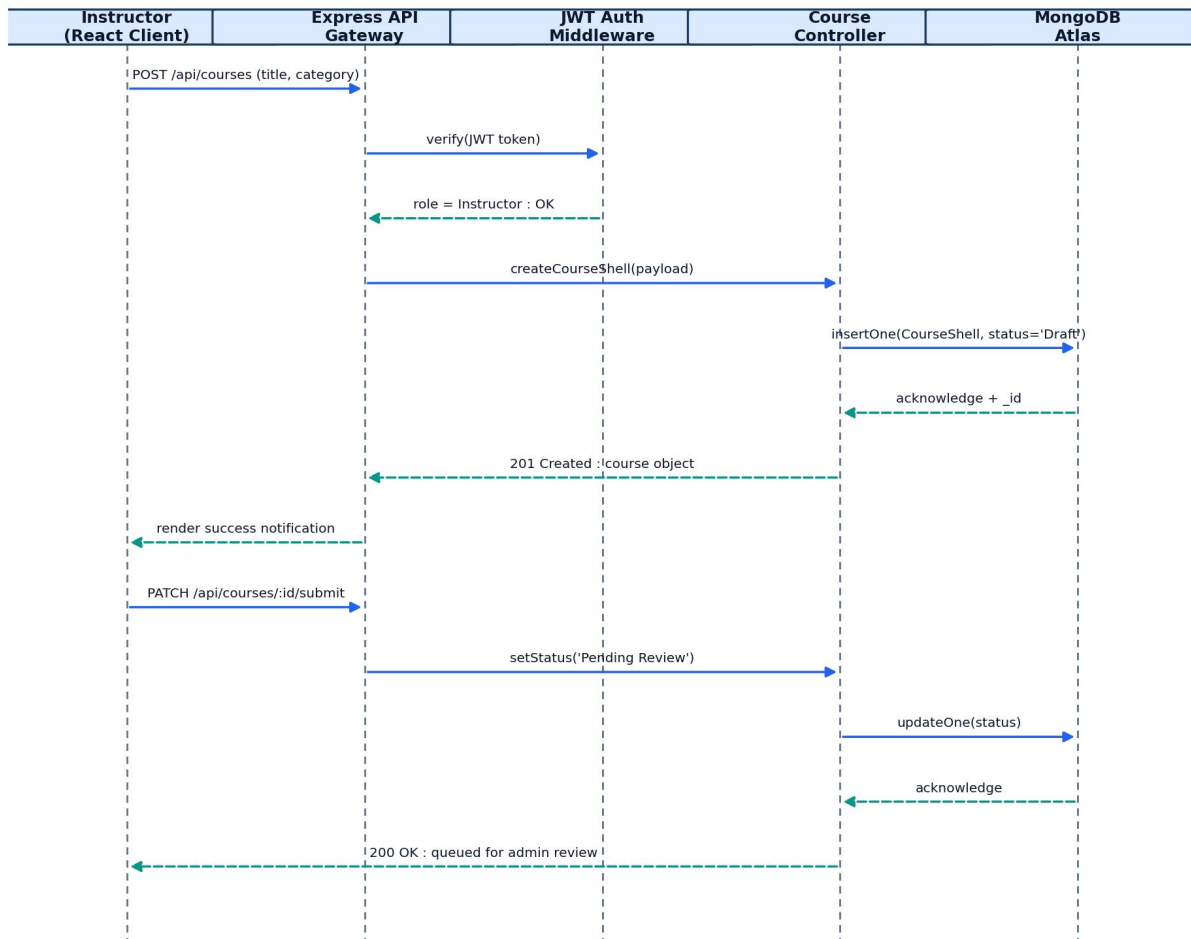


Figure 2.2: Sequence Diagram — Multi-Role Course Creation Request Workflow

Figure 2.3: Sequence Diagram — Timed Quiz Submission and Real-Time Grading Workflow

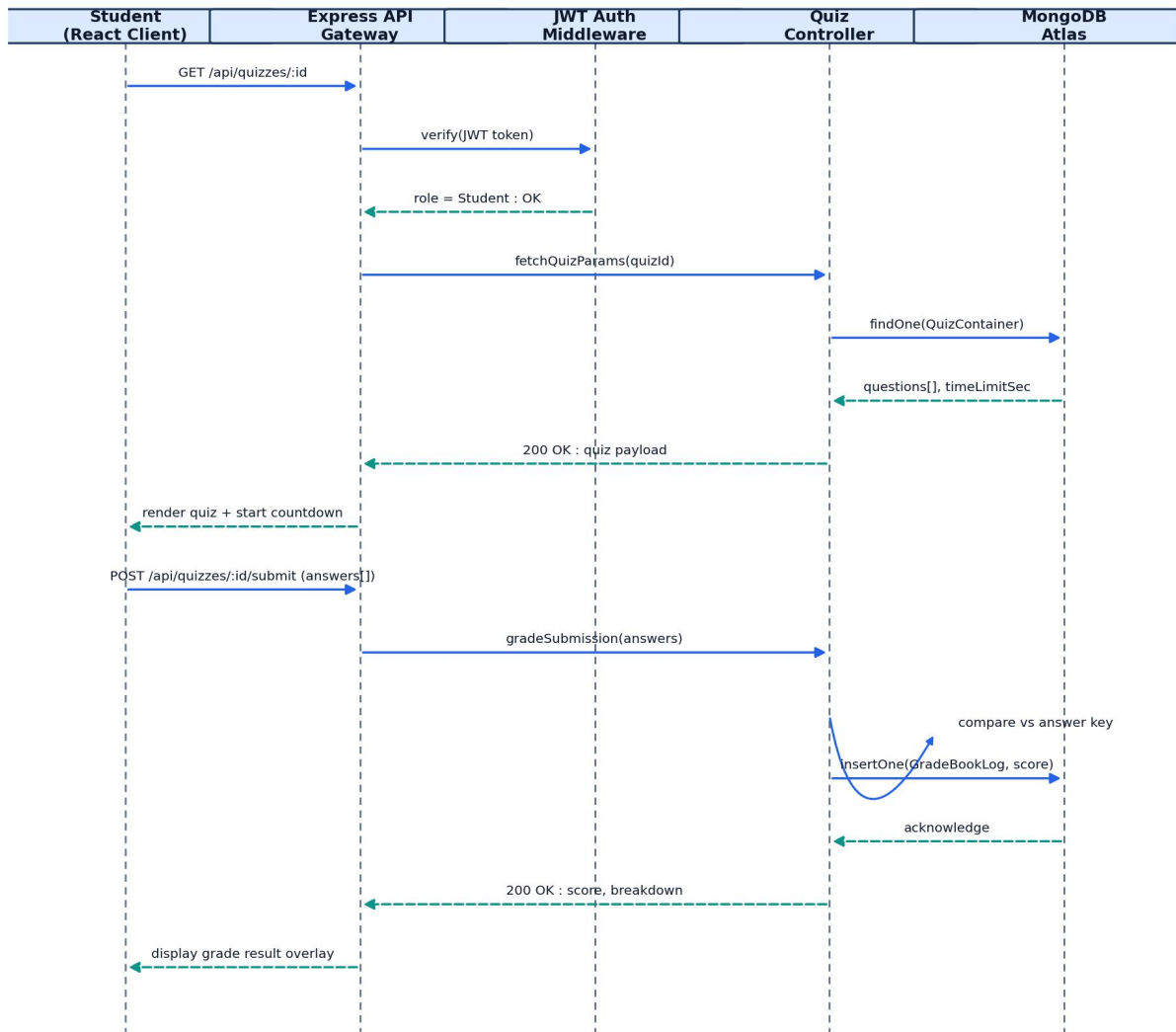


Figure 2.3: Sequence Diagram — Timed Quiz Submission and Real-Time Grading Workflow

Figure 2.2b: Sequence Diagram — Student Course Enrollment Workflow

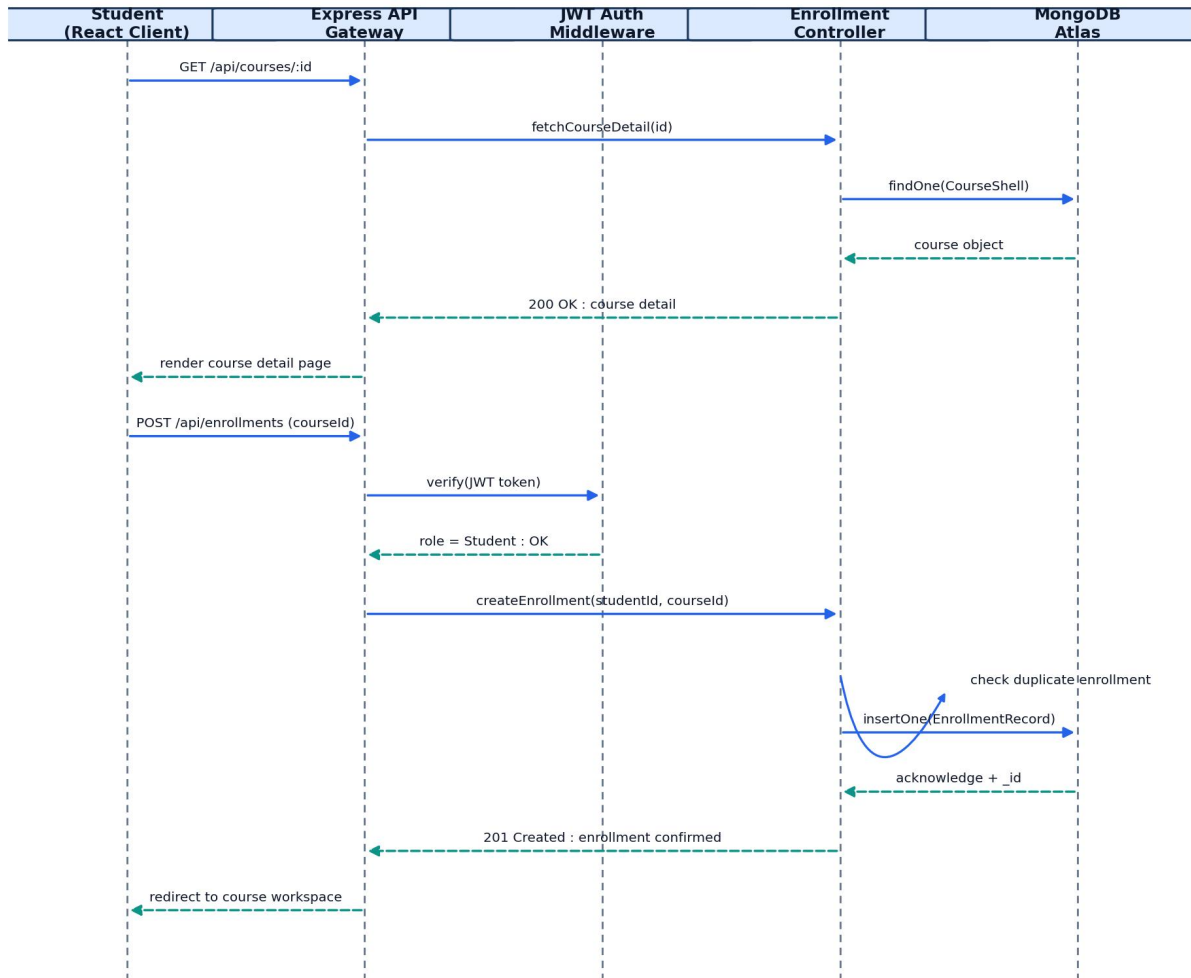


Figure 2.2b: Sequence Diagram — Student Course Enrollment Workflow

Figure 2.2c: Sequence Diagram — Assignment Submission and Instructor Grading Workflow

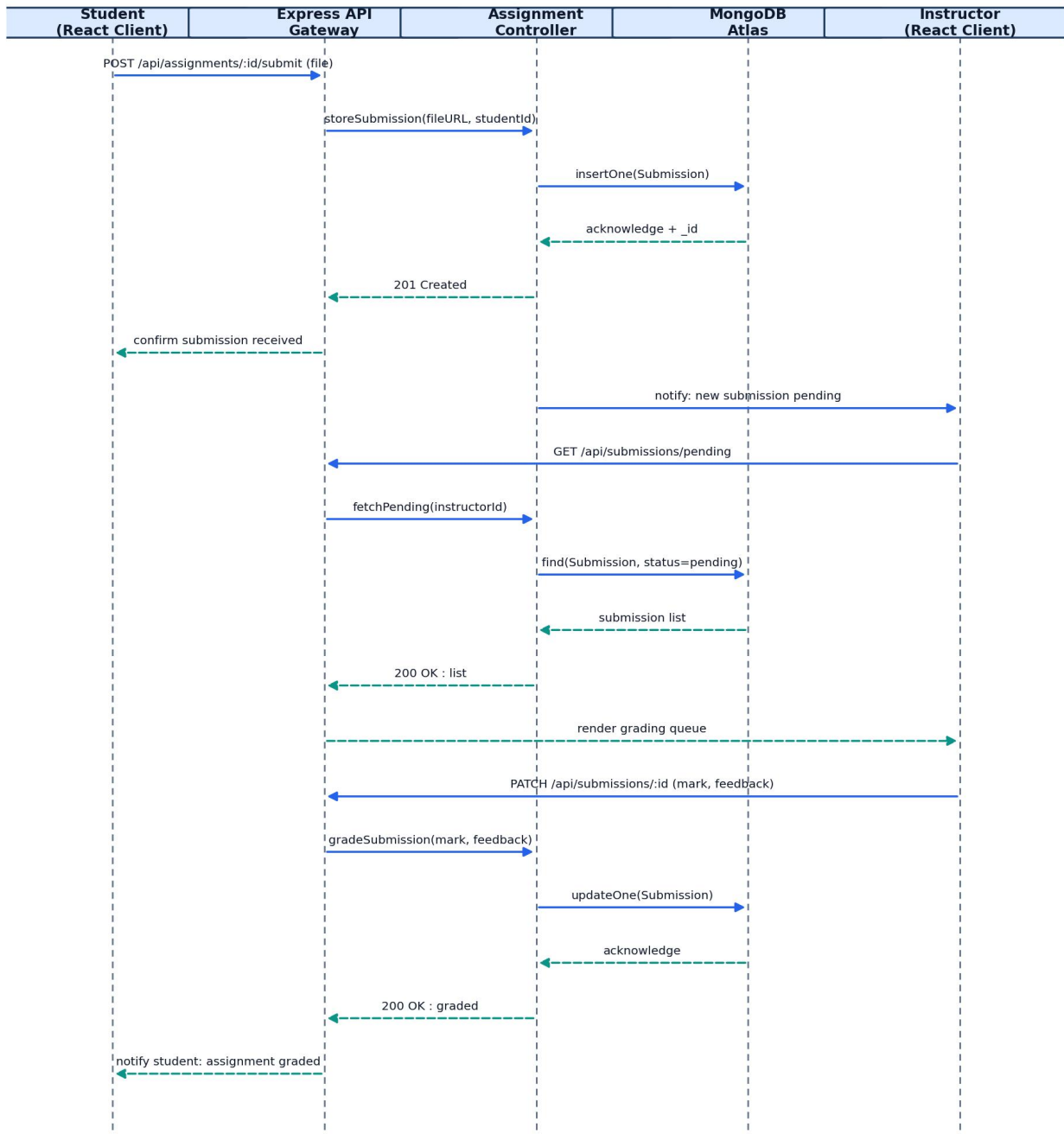


Figure 2.2c: Sequence Diagram — Assignment Submission and Instructor Grading Workflow

2.2.6 State Chart Diagram

The State Chart Diagram models the behavioral lifecycle transitions of the Course Shell entity within the MongoDB cluster. A course document exists in one of five mutually exclusive states, with transitions governed by explicit authorization protocols:

- **Draft State:** The initial state when an instructor creates a course shell. The document is completely hidden from the student catalog, allowing the creator to freely add, update, or remove text parameters and lesson placeholders.
- **Pending Review State:** Triggered when the instructor clicks 'Submit for Review'. The system locks the course contents, blocking instructor alterations while pushing a verification token to the Administrator's Audit Interface.
- **Published State:** Triggered exclusively when an administrator provides a formal clearance flag. The course configuration shifts to an active state, making it immediately visible within the student catalog and unlocking enrollment endpoints.
- **Rejected / Revision Needed State:** If the administrator finds structural errors, missing content, or low quality materials, the course transitions to a revision state. This unlocks modification privileges for the instructor while keeping the course hidden from students.
- **Archived State:** When a training cycle ends, the administrator can retire the package. This moves the course to an archived state, preserving grade historical records for graduated students while blocking new user enrollment.

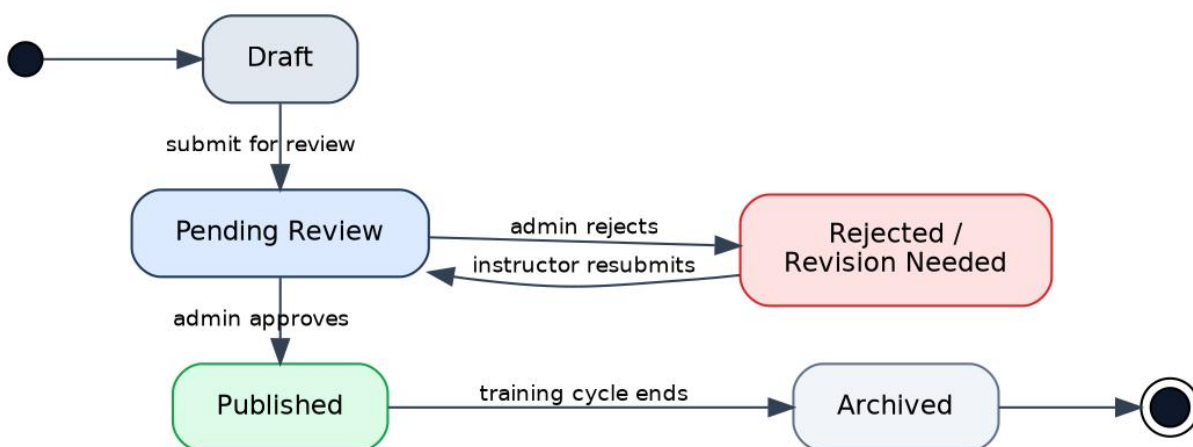


Figure 2.4: State Chart Diagram — Course Shell Entity Lifecycle Transitions

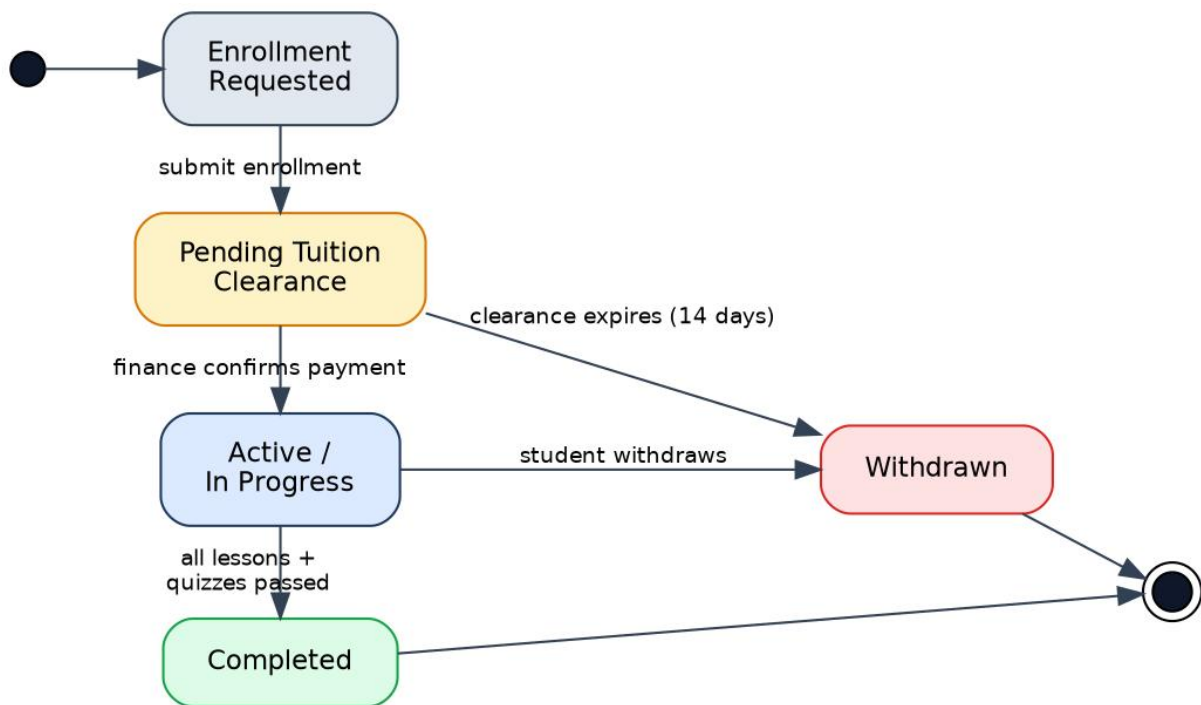


Figure 2.4b: State Chart Diagram — Enrollment Record Lifecycle Transitions

2.2.7 Activity Diagram

The Activity Diagram uses a multi-swimlane format to model the parallel processing paths, conditional decision bifurcations, and iterative operational loops within the platform's core modules. The diagram below illustrates the Student Lesson Media Streaming Authentication Route.

When a student logs into the portal and requests to stream a video module, the system initiates a parallel processing check: the API Gateway validates the authentication token, and if valid, the data validation layer checks the student's financial clearance status. Only students with confirmed tuition clearance and valid session tokens are granted access to the CDN video streaming pipeline.

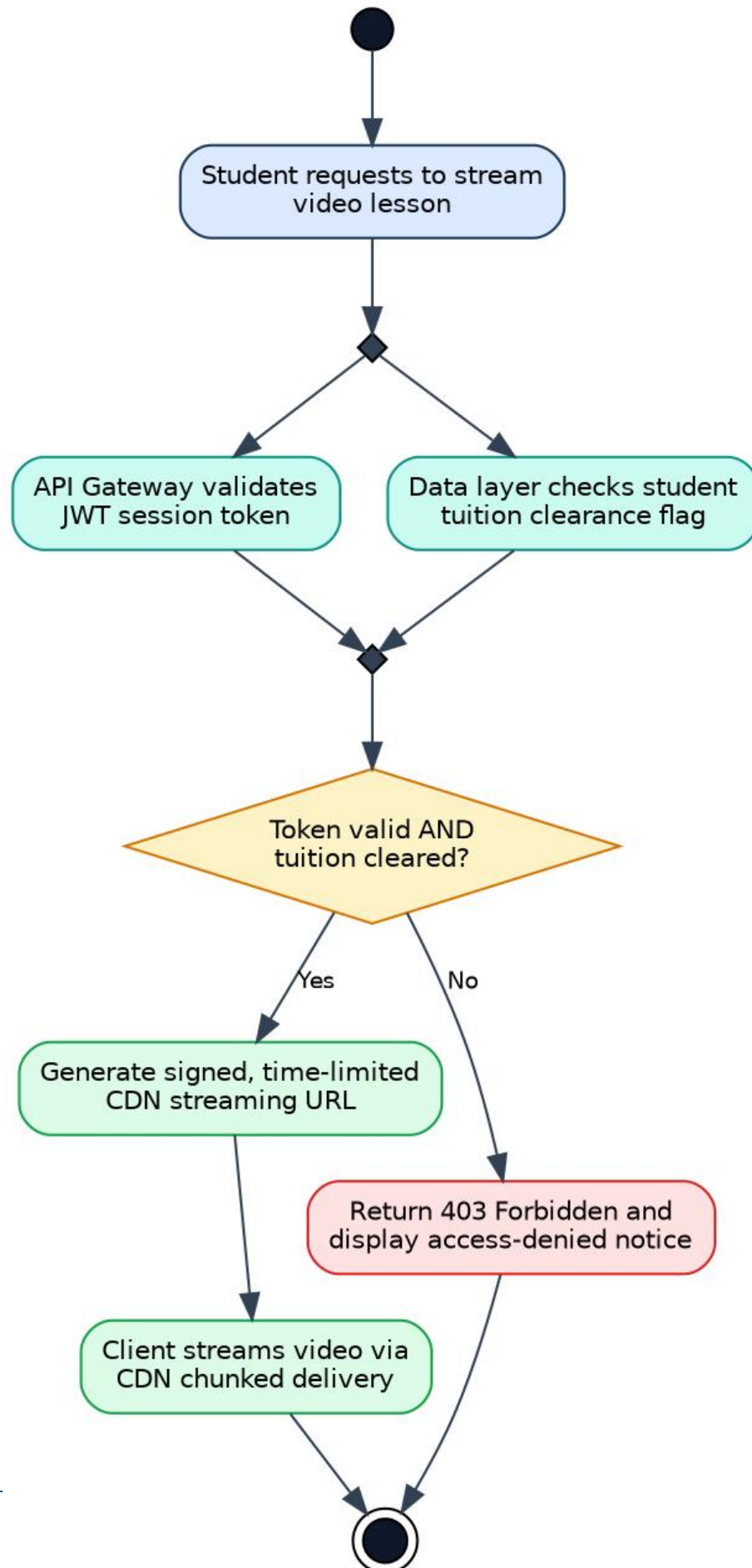


Figure 2.5: Activity Diagram — Student Lesson Media Streaming Authentication Route

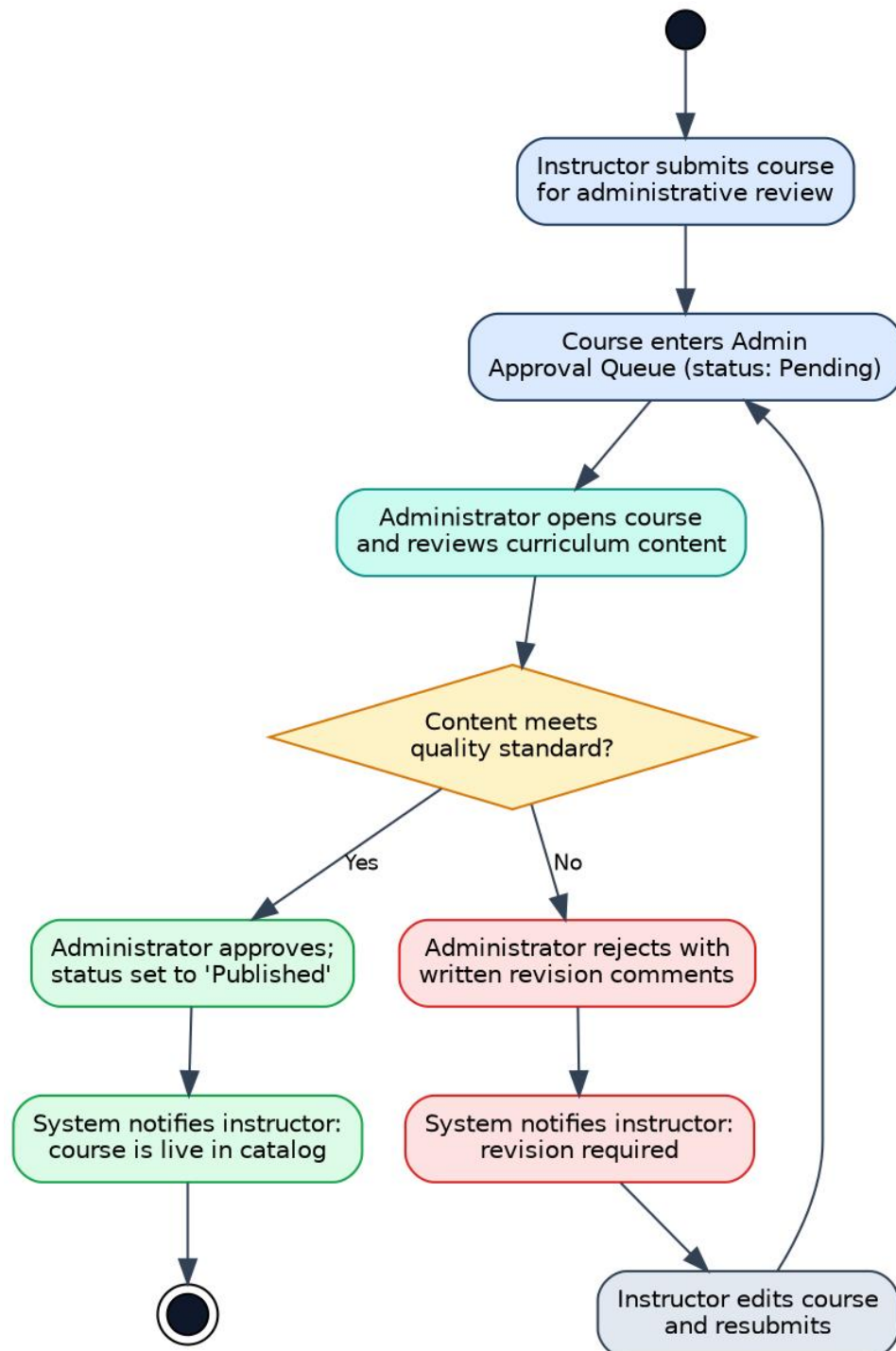


Figure 2.5b: Activity Diagram — Administrator Course Approval Workflow

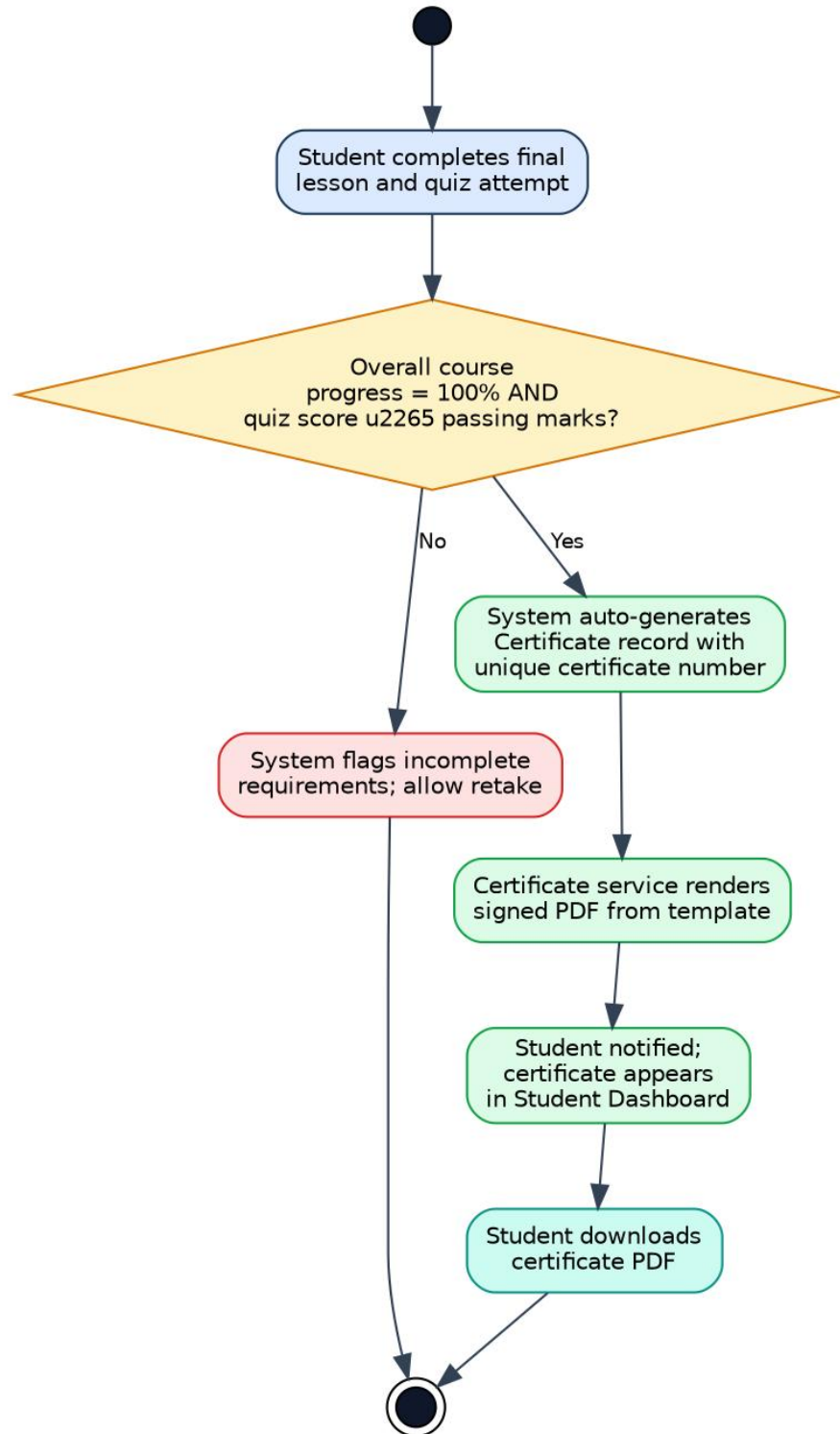


Figure 2.5c: Activity Diagram — Course Completion and Certificate Generation

2.3 Key Abstraction with CRC Analysis

To bridge the gap between behavioral use case requirements and structural class design architectures, the domain space is abstracted into definitive software objects using Class-Responsibility-Collaborator (CRC) modeling. This technique groups system components based on their logical data encapsulation zones and facilitates the identification of object interactions before class diagram construction.

Class Name	Responsibilities	Collaborators
UserAccount	- Store user identity attributes and cryptographic credentials - Manage session authentication and token lifecycle - Enforce role-based access boundaries	- InstructorProfile - StudentProfile - CourseShell - NotificationEngine
CourseShell	- Encapsulate course metadata and publication state - Manage nested chapter and lesson array structures - Control enrollment endpoint activation	- UserAccount (Admin, Instructor) - CurriculumLesson - QuizContainer - EnrollmentRecord
CurriculumLesson	- Encapsulate instructional video streaming URLs - Store associated markdown source references - Track structural sequence location indices - Update student viewing permission parameters	- CourseShell - ProgressTracker - UserAccount
QuizContainer	- Manage assessment question arrays and answer keys - Enforce countdown timer parameters - Calculate final score metrics upon submission	- CourseShell - GradeBookLog - StudentProfile
GradeBookLog	- Store atomic numerical score points per assessment - Compile qualitative text reviews from instructors - Compute cumulative course completion averages - Maintain chronological performance logs	- UserAccount - QuizContainer - EnrollmentRecord
EnrollmentRecord	- Track student-course enrollment pairs - Monitor cumulative completion percentage - Store tuition clearance flags per course	- UserAccount (Student) - CourseShell - ProgressTracker
Notification-engine	- Monitor system transnational data event triggers - Construct and broadcast automated web alert streams - Dispatch grade publication and deadline alerts	- UserAccount - GradeBookLog - CourseShell

Figure 2.6: CRC Cards — Key System Abstractions

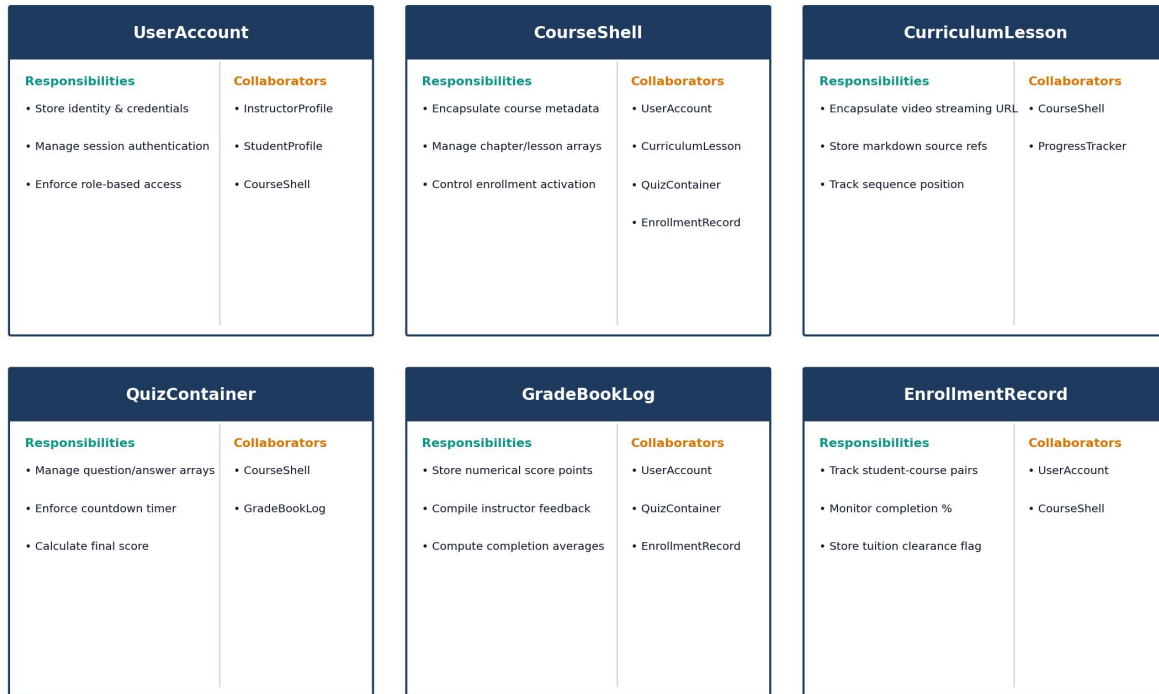


Figure 2.6: CRC Cards — Key System Abstractions

2.4 Class Diagram

The static structural design of the platform's object models is formally specified in the UML Class Diagram. It explicitly enforces type designations, property restrictions, visibility definitions (- for private fields, + for public methods), and relational associations including single and multi-node composition links and structural generalization paths.

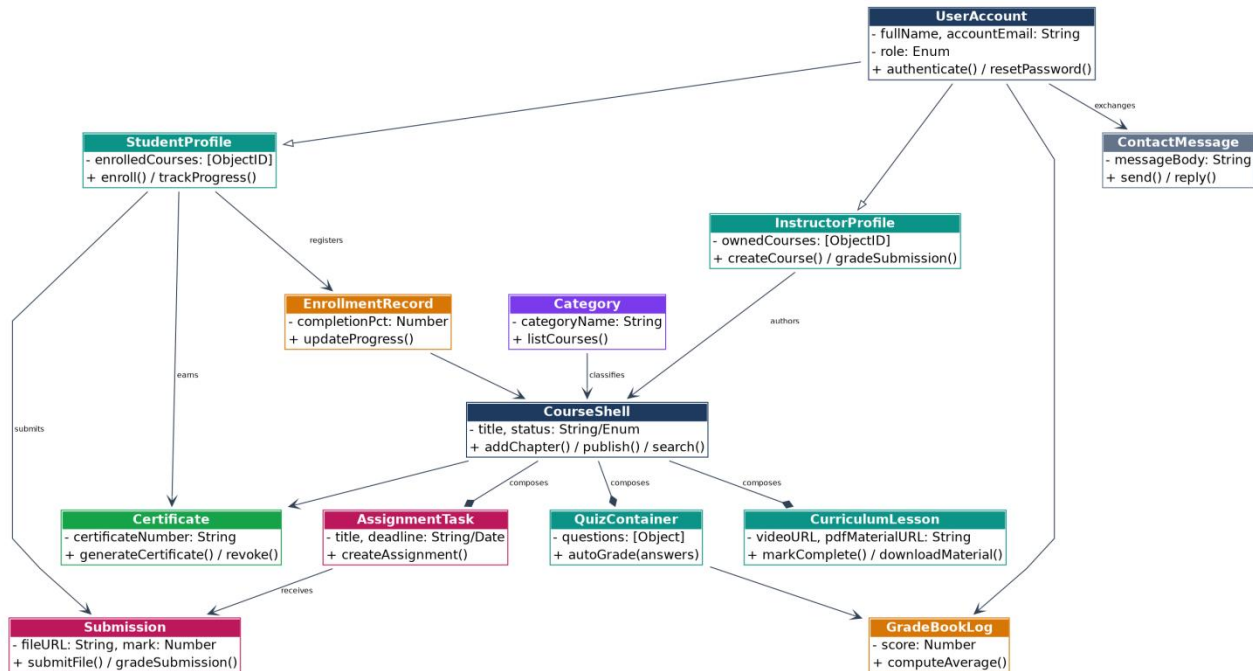


Figure 2.7: Class Diagram — Emare E-Learning Platform Object Model

The class diagram presents six primary object classes with the following key relationships:

- User-account is the parent class with generalization relationships to Student Profile and Instructor Profile, modeling role-specific behavioral extensions.
- Student Profile has a 1-to-many enrollment association with Course-shell, representing course registrations.
- Instructor Profile has a 1-to-many authorship association with Course-shell, representing course ownership.
- Course-shell has composition relationships with Curriculum Lesson and Quiz Container, meaning lessons and quizzes cannot exist independently of a parent course.
- GradeBookLog maintains association links to both User-account (for student identification) and Quiz-container (for assessment reference tracking).

2.5 Identifying Change Cases

Change cases describe potential system evolution scenarios that must be considered during architectural design to ensure the platform remains adaptable to future operational shifts:

Change Case ID	Description	Likelihood	System Impact Level
CC-01	Integration of AI-Assisted Code Assessment: Connecting external large language model endpoints to automatically scan student programming submissions for syntax quality, security vulnerabilities, and formatting compliance before instructor manual verification.	High — E mare's strategic focus on AI innovation makes this a near-term priority.	Low to Medium — Clean Controller-Service-Repository abstraction allows isolated AI service module insertion without disrupting core quiz or student profile modules.
CC-02	Offline-First Local Network Sync Architecture: Deploying in-house synchronization servers inside the Debre_Birhan lab to enable students to stream content and complete assessments during internet outages, with automatic cloud sync on re-connection.	Medium — Internet reliability in regional Ethiopia remains variable.	High — Requires Pouch Db or Couch-db synchronization engines on the client layer and conflict-resolution logic within the Mongo_DB Atlas data ingestion tier.
CC-03	Muti-Language Interface Support: Adding Amharic and Afaan Oromo language localization options to the user interface to increase accessibility for instructors and students with limited English proficiency.	Medium — Strong alignment with national digital inclusion policies.	Low — React internationalization libraries (i18n) can be integrated with minimal disruption to existing component architecture.
CC-04	Real-Time Live Collaboration Rooms: Integration of WebRTC protocols to allow students to run live code-pairing sessions, share screens, and access interactive virtual whiteboards inside the browser dashboard.	Medium — Demand for synchronous remote learning is growing.	High — Requires dedicated WebRTC signaling server infrastructure and major frontend component additions.

2.6 User Interface Prototyping

The functional layouts of the application's responsive view components are structurally mapped out in the following prototypes. These mockups establish the core UI/UX patterns for the development team before active frontend component construction begins, ensuring alignment between user requirements and final implementation.



Figure 2.8: UI Prototype — Student Split-Screen Workspace Layout (video panel + persistent nested curriculum sidebar)

Figure 2.8: UI Prototype — Student Split-Screen Workspace Layout

UI PROTOTYPE (LOW-FIDELITY WIREFRAME)

EVALUATION PORTAL — Grade Submission **Course: Full-Stack Web Development**

Student

Selamawit Tesfaye — Cohort B — Submitted 2 days ago

Deliverable

github.com/selamawit-t/final-capstone-api [Open]

Score (/ 100) Pass / Fail Threshold

87 ✓ Passed (≥ 60)

Qualitative Feedback

Strong REST API structure and clean error handling. Add input validation on the enrollment endpoint before resubmission of future assignments.

Save Draft Publish Grade Record

Figure 2.9: UI Prototype — Instructor Assessment Feedback Form Layout

Figure 2.9: UI Prototype — Instructor Assessment Feedback Form Layout

CHAPTER THREE

OBJECT-ORIENTED SYSTEM DESIGN

3.1 Purpose and Goals of Design

System design translates the functional and non-functional requirements specified in Chapter Two into concrete architectural decisions, component structures, and data models. The design phase serves as the critical bridge between what the system must do (requirements) and how the system will be built (implementation). For the Emare E-Learning Management System, the following design goals guide all architectural decisions:

Performance and High Throughput

The platform must maintain a rapid execution envelope to support heavy interactive use across Debre Birhan and surrounding regions. Standard database reads, metadata queries, and student progress saves must execute with sub-500ms server response latencies under concurrent loads of up to 500 users. Media delivery pipelines must optimize video stream rendering through CDN data chunking methods to achieve immediate playback with a maximum buffering timeout of 2 seconds over standard mobile broadband networks.

Dependability and Data Fault Tolerance

Data consistency and high availability are essential for a remote learning platform. The system architecture must guarantee an operational runtime availability threshold of 99.5%, implementing automated incremental nightly database collection backups. It must protect data from system failures by routing database errors through an Express.js global error-handling layer that logs structural tracking traces to a secure server directory while keeping the user interface active with clean, non-revealing error alerts.

Security and Information Governance

The platform must enforce strict role-based access control maps across distinct administrative, instructor, and student layers. It must cryptographically isolate operational workflows so that no user can manipulate or fetch documents outside their authorized access levels. All persistent credentials

must be encrypted using the bcrypt hashing algorithm with a security work factor threshold of 10, and all sessions must automatically expire after 120 minutes of inactivity.

Maintainability and Architectural Extensibility

The platform codebase must be highly modular to accommodate future system expansions. Following an object-oriented paradigm, components must feature low architectural coupling and clear separation of concerns across route controllers, business logic services, and database schemas. This ensures developers can easily integrate future modules such as real-time video streaming or AI assessment tools without rewriting core operational engines.

3.2 Current Software Architecture

Prior to this project, Emare ICT Hub had no formal software architecture. The existing training delivery model operated entirely on an ad-hoc, manual basis without any standardized digital systems framework. Content resided on individual instructor laptops, grades were tracked in disconnected spreadsheets, and student communications were managed through unstructured messaging channels. This absence of a unified software architecture is precisely the gap this project addresses.

3.3 Proposed Software Architecture — Subsystem Decomposition

To translate the design goals into a functional software system, the application is organized using a decoupled, three-tier MERN architecture. Each tier operates within a defined boundary, communicating via asynchronous JSON-based RESTful network APIs.

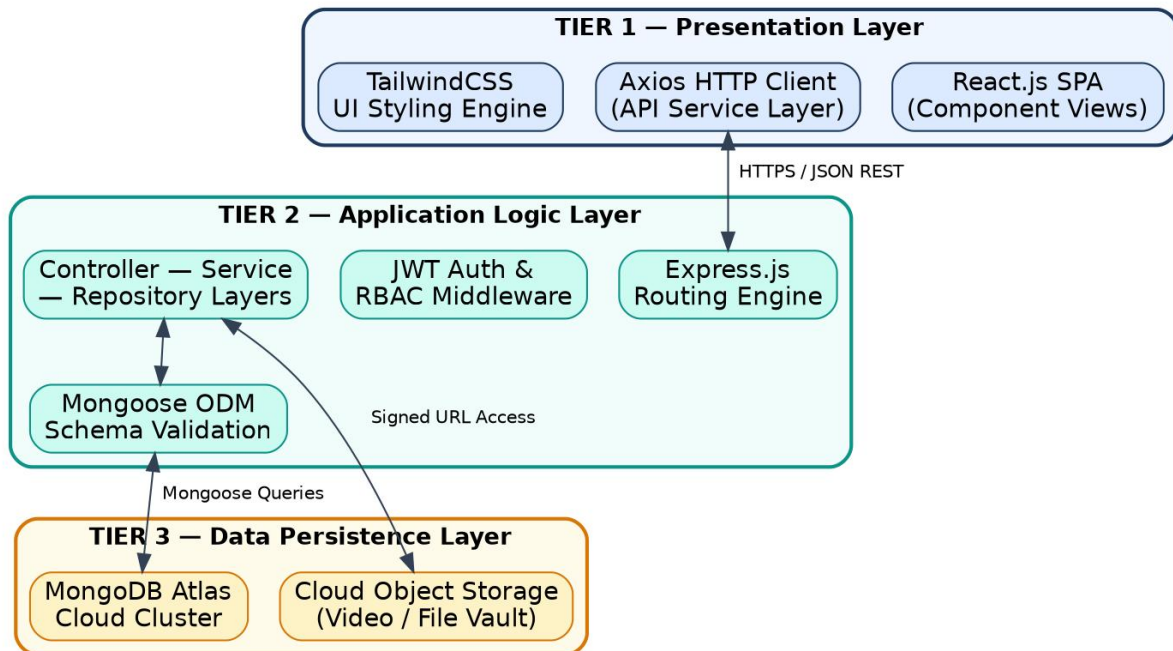


Figure 3.1: Three-Tier MERN Architecture Subsystem Decomposition

Tier 1 — Presentation Layer (React.js Frontend)

Built on the React.js single-page library, this layer manages all user-facing interactions. It maintains state machines for the user interfaces, validates input strings client-side, stores session tokens securely in browser-accessible storage, and triggers real-time visual adjustments via reactive component renders. The presentation layer communicates with the application logic tier exclusively through HTTPS-secured Axios API calls, maintaining a clean separation from business logic operations.

Tier 2 — Application Logic Layer (Node.js + Express.js Backend)

Operating on the Node.js asynchronous runtime and utilizing the Express.js routing engine, this tier processes the business logic of the system. It decrypts incoming network headers, enforces JSON schema validations, manages role-based verification layers through JWT middleware, routes computational workflows to controller modules, and connects directly to database endpoints through the Mongoose ODM layer. The backend follows a clean Controller-Service-Repository abstraction pattern to maintain maximum code modularity.

Tier 3 — Data Persistence Layer (MongoDB Atlas)

Managed via a secure MongoDB Atlas cloud database instance, this layer encapsulates the persistent system state. It stores platform metrics, logs grading matrices, maintains media streaming URLs, and executes indexed data read/write operations. The document-oriented nature of MongoDB perfectly matches the hierarchical data structures required for nested course curricula, embedded lesson arrays, and complex student progress tracking.

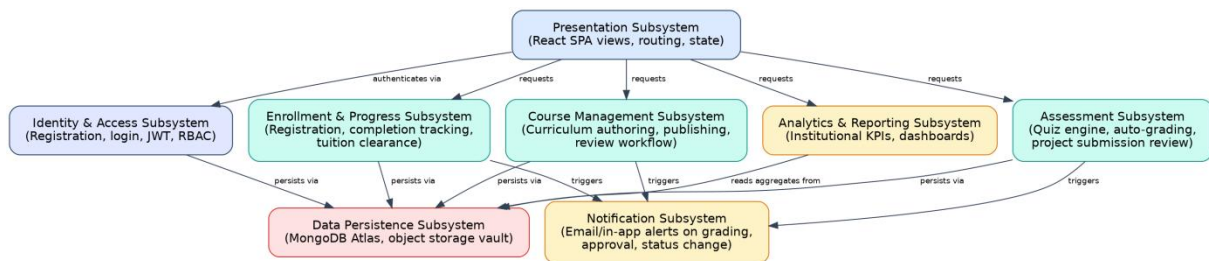


Figure 3.1b: Subsystem Decomposition Diagram — Service Coupling and Coherence

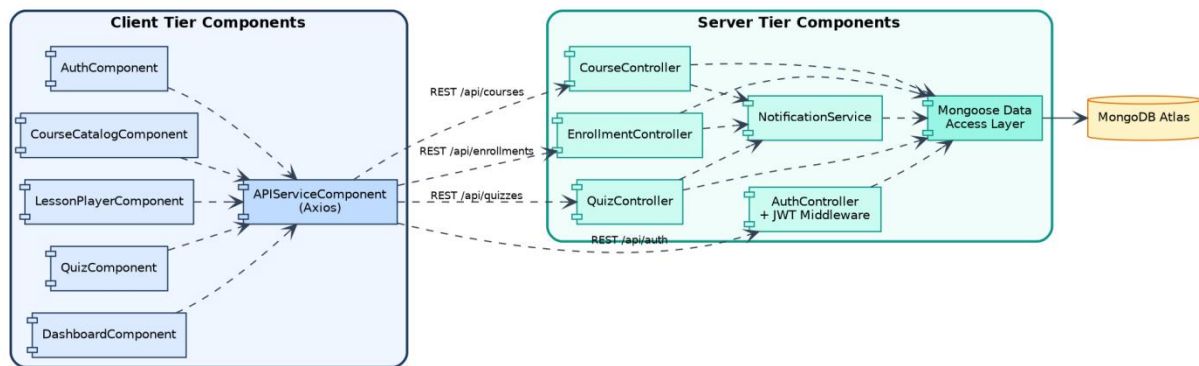


Figure 3.1c: Component Diagram — Client and Server Component Grouping

3.4 Deployment Diagram

The deployment diagram illustrates the physical infrastructure layout defining the production environment where the MERN software tiers operate. It balances performance, cost-efficiency, and system availability.

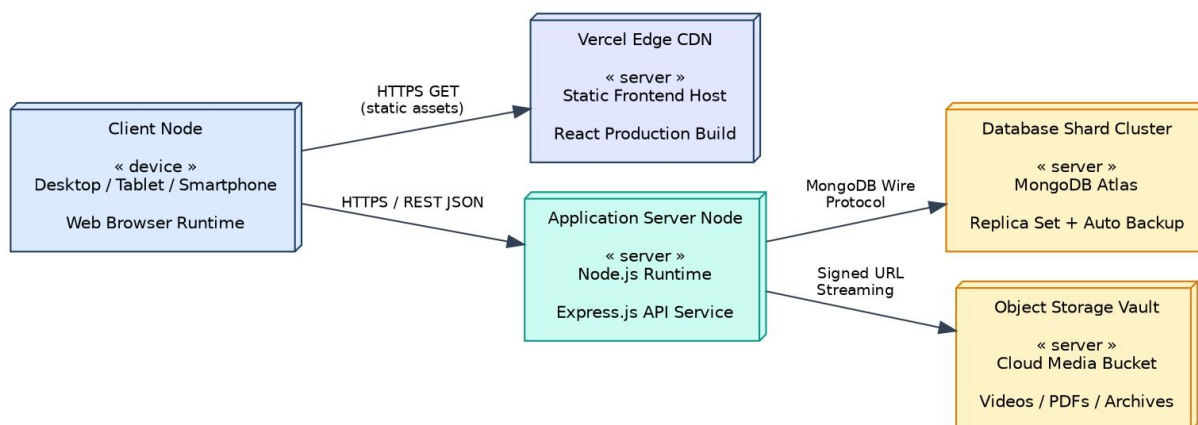


Figure 3.2: Hardware Deployment Diagram — Production Environment

User Client Nodes

Represent the personal devices (desktops, laptops, tablets, and smart phones) used by students, instructors, and administrators to access the system via standard modern web browsers. The static front-end React build assets are distributed globally via high-speed Vercel edge CDN networks to ensure fast load times across different network environments.

Application Server Node

A hosted virtual server container running the Node.js production service environment. This engine listens for incoming HTTPS requests on the configured port, processes data computations through the Express routing framework, validates security headers, and routes database traffic through the Mongoose ODM connector.

Persistent Database Shard Cluster

An isolated cloud data system running on MongoDB Atlas. It organizes data records across multiple geographic cluster nodes to maintain linear look-up speeds, fast indexing performance, automated data backups, and fail-over redundancy.

Object Storage Media Vault

A dedicated cloud media storage bucket that holds high-definition instructional video lectures, code zip archives, and PDF documentation manuals. The application server streams files securely from this storage vault to authorized users through signed URL generation with time-limited access tokens.

3.5 Database Design — Persistence Modeling

Because the platform utilizes Mon go_DB, data modeling leverages a document-oriented structure. Relationships between collections are managed using explicit Object ID foreign key references, ensuring referential integrity while maintaining Mon go_DB flexible schema advantages.

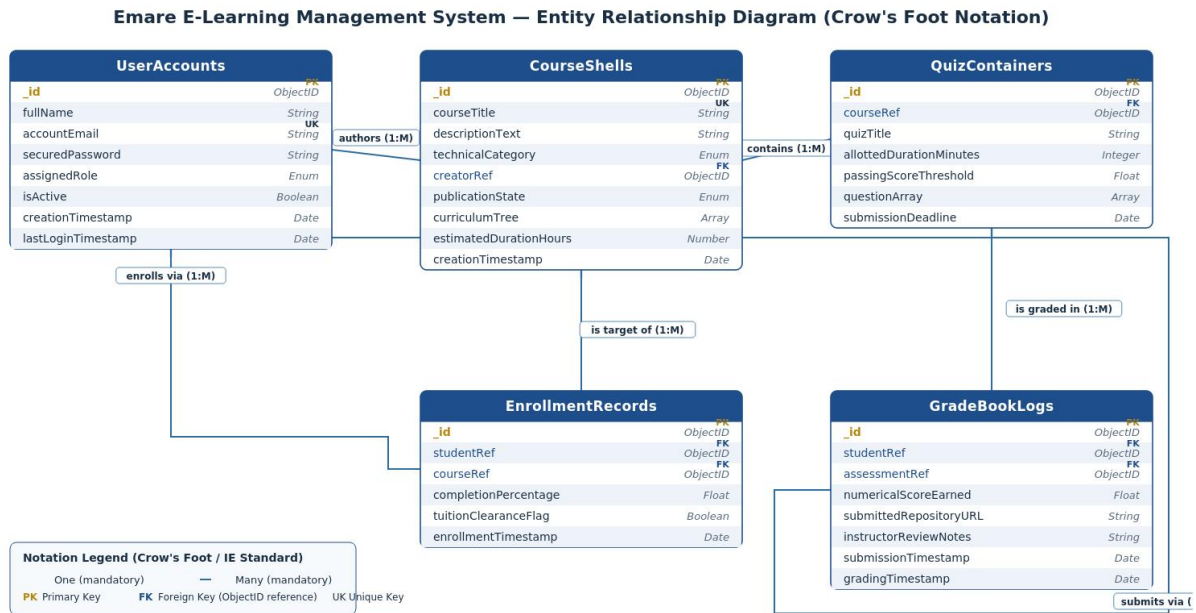


Figure 3.3: Entity Relationship Diagram of the E mare ELMS Data Model (Crow's Foot Notation)

Collection 1: UserAccounts

Field Name	Data Type	Constraints
_id	ObjectID	Primary Key, Auto-generated, Unique Index
fullName	String	Required, 2 to 100 characters
accountEmail	String	Required, Unique, Validated against email regex pattern
securedPassword	String	Required, Minimum 60 characters (bcrypt hash output)
assignedRole	String (Enum)	Values: 'Admin', 'Instructor', 'Student' — Required
isActive	Boolean	Default: true; false suspends all access
creationTimestamp	Date	Default: Date.now() at creation time
lastLoginTimestamp	Date	Updated on every successful authentication

Collection 2: CourseShells

Field Name	Data Type	Constraints
_id	ObjectID	Primary Key, Auto-generated, Unique Index
courseTitle	String	Required, Unique, 5 to 150 characters
descriptionText	String	Required, Minimum 20 characters
technicalCategory	String (Enum)	Values: 'Web Coding', 'Creative Media', 'Robotics Hardware', 'Network Engineering', 'Mobile Development'
creatorRef	ObjectID	Foreign Key referencing User-accounts._id — Required
publication State	String (Enum)	Values: 'Draft', 'Pending Audit', 'Active', 'Archived' — Default: 'Draft'
curriculumTree	Array of Objects	Contains sequential Chapter sub-documents with nested lesson arrays
estimatedDurationHours	Number	Required, represents total course length in hours
creationTimestamp	Date	Default: Date.now()

Collection 3: QuizContainers

Field Name	Data Type	Constraints
_id	ObjectID	Primary Key, Auto-generated, Unique Index
courseRef	ObjectID	Foreign Key referencing CourseShells._id — Required
quizTitle	String	Required, descriptive title of the assessment
Allotted-duration Minutes	Integer	Required, Min: 5 minutes, Max: 180 minutes
passingScoreThreshold	Float	Default: 60.0 — minimum percentage to pass
questionArray	Array of Objects	Contains Question text, option arrays, and correct answer indices
submissionDeadline	Date	Required, enforces late submission penalty rules

Collection 4: EnrollmentRecords

Field Name	Data Type	Constraints
_id	ObjectID	Primary Key, Auto-generated, Unique Index
studentRef	ObjectID	Foreign Key referencing User-accounts._id — Required
courseRef	ObjectID	Foreign Key referencing Course-shells._id — Required
completionPercentage	Float	Range: 0.00 to 100.00 — updated by video completion triggers
Tuition ClearanceFlag	Boolean	Default: false — manually confirmed by Administrators
enrollmentTimestamp	Date	Default: Date.now() at enrollment

Collection 5: GradeBookLogs

Field Name	Data Type	Constraints
_id	ObjectID	Primary Key, Auto-generated, Unique Index
studentRef	ObjectID	Foreign Key referencing User-accounts._id — Required
assessmentRef	ObjectID	Foreign Key referencing Quiz-containers._id or Assignment ID
Numerical Score-earned	Float	Range: 0.00 to 100.00 — computed or manually assigned
submittedRepositoryURL	String	Optional — valid URI pointing to Git repository or file upload
instructorReviewNotes	String	Optional — qualitative feedback text block from instructor
submissionTimestamp	Date	Default: Date.now() at submission time
gradingTimestamp	Date	Populated when instructor publishes the grade record

3.6 Access Control and Security

To maintain strict run-time information security, endpoint permissions are organized via a multi-layered verification matrix. This map determines exactly which user roles can access specific system resources and what database operations are permitted.

API Route Endpoint	HTTP Method	Student	Instructor	Admin	Database Action
/api/users/provision	POST	Denied	Denied	Granted	Writes new user profile document
/api/courses/draft	POST	Denied	Granted	Denied	Pushes new parent course model with Draft status
/api/courses/audit-clear	PATCH	Denied	Denied	Granted	Mutates course status to Active
/api/lessons/stream/:id	GET	Conditional*	Granted	Granted	Streams target video chunk file path via CDN
/api/quizzes/evaluate	POST	Granted	Denied	Denied	Logs score data to GradeBook collection
/api/grades/publish	POST	Denied	Granted	Denied	Commits manual grading metrics to database
/api/admin/metrics	GET	Denied	Denied	Granted	Aggregates platform-wide performance statistics
/api/enrollments/confirm	PATCH	Denied	Denied	Granted	Sets tuition Clearance-flag to true for student

*Conditional: Student access requires tuition Clearance-flag = true and valid JWT session.

3.7 RESTful API Endpoint Documentation

To formally document how the React.js front-end communicates with the Node.js /Express back-end, the platform's primary REST endpoints are summarized below. All endpoints (except registration and login) require a valid JWT bearer token, and access is further constrained by the Role-Based Access Control matrix defined in Table 3.5.

Endpoint	Method	Access Role	Description
/api/auth/register	POST	Public	Creates a new user account with hashed password
/api/auth/login	POST	Public	Authenticates credentials and issues a JWT session token
/api/courses	GET	All Roles	Retrieves the list of published courses
/api/courses	POST	Instructor	Creates a new course shell pending admin approval
/api/courses/:id/approve	PATCH	Administrator	Approves a submitted course for publication
/api/courses/:id/enroll	POST	Student	Enrolls the authenticated student in a course
/api/lessons/:id/stream	GET	Student	Streams a video lesson (requires tuition clearance)
/api/quizzes/:id/attempt	POST	Student	Submits timed quiz answers for automatic grading
/api/submissions	POST	Student	Uploads a project file and repository URL
/api/submissions/:id/grade	PATCH	Instructor	Records a grade and feedback for a submission
/api/users	GET	Administrator	Retrieves platform-wide user accounts
/api/analytics/overview	GET	Administrator	Retrieves aggregate enrollment and performance analytics

Table 3.6: Restful API Endpoint Summary

Full request/response sachems, query parameters, and error codes for each endpoint are maintained in the project's Postman collection and are version alongside the back-end repository referenced in Appendix C.

CHAPTER FOUR

IMPLEMENTATION, TESTING, AND MAINTENANCE

4.1 Introduction to Languages, IDEs, Tools, and Technologies

The implementation phase translates the architectural designs from Chapter Three into working, executable code. The choice of technologies reflects both the technical requirements of the system and the practical expertise of the development team at Emare ICT Hub.

Technology / Tool	Category	Version	Purpose
React.js	Frontend Framework	18.x	Single-page application UI development with component-based architecture
TailwindCSS	CSS Framework	3.x	Utility-first responsive styling for cross-device compatibility
Node.js	Backend Runtime	LTS 20.x	Asynchronous server-side JavaScript execution environment
Express.js	API Framework	4.x	RESTful API routing, middleware management, and HTTP handling
MongoDB Atlas	Database	7.x Cloud	Document-oriented NoSQL data persistence with cloud clustering
Mongoose	ODM Library	8.x	Schema-based MongoDB object modeling and validation
bcrypt	Security Library	5.x	Password hashing with configurable salt work factor
jsonwebtoken	Auth Library	9.x	JWT token generation, signing, and verification
Axios	HTTP Client	1.x	Promise-based HTTP requests from React frontend to Express API
Vercel	Deployment Platform	Latest	Frontend static asset hosting with global CDN distribution
Visual Studio Code	IDE	Latest	Primary code editor with ESLint and Prettier extensions
StarUML	Modeling Tool	5.x	UML diagram creation and system design documentation
Postman	API Testing	Latest	Manual and automated REST API endpoint testing

Technology / Tool	Category	Version	Purpose
Git / GitHub	Version Control	Latest	Source code versioning, collaboration, and backup management

4.2 Coding Standards of the Languages Used

Directory Structure Pattern

The backend codebase follows a strict Controller-Service-Repository abstraction model organized as follows:

- routes/ — Router files exclusively capture endpoint strings and delegate parameters to controller layers.
- controllers/ — Controller classes orchestrate HTTP response cycles and delegate business logic to service modules.
- services/ — Service modules contain all business logic computations and communicate with the database through Mongoose schema models.
- models/ — Mongoose schema definitions enforcing strict data type validations and collection structure.
- middleware/ — Authentication, authorization, and error-handling middleware functions.
- config/ — Environment variables and database connection configuration files.

Variable Naming and Code Conventions

- camelCase formatting for all JavaScript variables, array names, function parameters, and method initializers.
- PascalCase for React component names and JavaScript class definitions.
- UPPERCASE_SNAKE_CASE for critical configuration constants and environment security keys.
- Asynchronous flow management uses modern async/await runtime wrappers encapsulated within try/catch middleware patterns to eliminate callback chain complexity.
- All API responses follow a standardized JSON structure: { success: Boolean, data: Object|Array, message: String } for consistent client-side handling.

4.3 Testing Techniques and Test Plans

Schedule of Activities and Gantt Chart

Phase	Activity	Duration	Weeks
Phase 1: Planning	Project scope definition, stakeholder interviews, requirements documentation	2 weeks	Weeks 1-2
Phase 2: Analysis	UML modeling, use case documentation, system feasibility analysis	2 weeks	Weeks 3-4
Phase 3: Design	Architecture design, database schema modeling, UI prototyping	2 weeks	Weeks 5-6
Phase 4: Backend Development	Node.js/Express API development, MongoDB integration, authentication system	3 weeks	Weeks 7-9
Phase 5: Frontend Development	React component development, UI implementation, API integration	3 weeks	Weeks 10-12
Phase 6: Testing	Unit testing, integration testing, UAT, security and performance testing	2 weeks	Weeks 13-14
Phase 7: Documentation	Final documentation, report writing, project submission preparation	1 week	Week 15

Budget Breakdown

Item	Description	Estimated Cost (ETB)
Internet Connectivity	High-speed internet for development, API testing, and cloud deployment	2,400
Cloud_Hosting (Vercel)	Frontend deployment and CDN distribution (Free tier)	0
MongoDB Atlas	Database cloud hosting (Free M0 cluster tier)	0
Development Devices	Laptop/desktop usage (existing equipment)	0
Software Licenses	All tools are open-source (VS Code, StarUML Community, Postman Free)	0
Printing_and Documentation	Report printing and binding costs	500
Miscellaneous	Transport for field interviews and stakeholder meetings	300
TOTAL		3,200 ETB

Test Plans

To verify the application's readiness according to departmental requirements, the platform undergoes extensive automated and manual test sequences across four core areas:

- **Unit Testing:** Validating isolated computational functions, utility helpers, and individual schema database hooks in complete isolation from the rest of the system.
- **Integration Testing:** Tracking the behavioral patterns of coupled architectural nodes, ensuring that incoming payload models transfer smoothly between network routers, authentication filter blocks, and data controller methods.
- **Security and Penetration Testing:** Actively attempting to compromise role boundaries, inject malicious input vectors, or submit expired token headers to verify the system's defensive mechanisms.
- **Boundary and Stress Testing:** Evaluating how database indexes and the Express execution engine perform when input bounds are violated or when the system experiences heavy concurrent simulation loads.

Comprehensive Test Suite Results

Test ID	Module	Input	Technique	Expected Output	Status
TS-01	AUTH-MOD	Plaintext password: WebDev#2026	Unit Testing	MongoDB stores a 60-character bcrypt hash. Plaintext is destroyed.	PASS
TS-02	AUTH-MOD	Malformed email: emare.hub	Validation Testing	UI blocks POST request and outputs 'Invalid Email Format' alert.	PASS
TS-03	AUTH-MOD	GET to /api/admin/metrics with missing token	Integration Testing	API middleware blocks request; returns HTTP 401 Unauthorized.	PASS
TS-04	AUTH-MOD	Student JWT token targeting POST /api/courses/draft	Security Testing	Gateway detects role mismatch; returns HTTP 403 Forbidden.	PASS
TS-05	CMS-MOD	Student requests video with tuitionClearanceFlag: false	Functional Testing	System blocks stream; returns HTTP 402 Payment Required.	PASS
TS-	CMS-	Nested chapter	Schema	Sub-document	PASS

Test ID	Module	Input	Technique	Expected Output	Status
06	MOD	update payload to existing course	Integrity	appended correctly via MongoDB \$push operation.	
TS-07	ASSESS-MOD	Student submits quiz answers before timer expires	Performance Testing	Grade computed and committed to database in under 200ms.	PASS
TS-08	ASSESS-MOD	Browser countdown timer hits 00:00 threshold	Robustness Testing	UI freezes instantly; auto-dispatches partial answer array.	PASS
TS-09	ASSESS-MOD	Score value 115 input into 100-point scale field	Boundary Testing	Controller blocks write; returns out-of-scale validation error.	PASS
TS-10	CMS-MOD	Admin issues course approval clearance flag	Functional Testing	Course status updates to Active; student catalog refreshes in real time.	PASS

CHAPTER FIVE

RESULTS AND DISCUSSION

5.1 User Interface Representation of the System

The development process successfully produced a functional, decoupled multi-role web platform that completely digitizes the training ecosystem of E mare ICT Hub. By transitioning institutional workflows away from manual tracking processes, physical real estate constraints, and unmanaged file distribution channels, the application establishes a secure, unified, and scalable environment for modern technical education delivery.

Administrators now monitor organizational performance metrics through an interactive analytics panel that aggregates real-time enrollment logs, class completion velocities, and graduation tallies across the hub's coding, graphics, and hardware training streams. Instructors construct multi-level course architectures, attach streaming lecture video assets directly to cloud storage, deploy timed assessments with configurable parameters, and grade project deliverables from a centralized evaluation workspace. Students navigate a structured course directory, view responsive video content through a split-screen interface, track their module completion metrics with visual progress indicators, and complete interactive evaluations with instant automated grade feedback.

5.1.1 Brief Description of System Modules

Module	Primary Actor	Key Functions
Authentication Module	All Users	Secure login with JWT session management, role detection, and session timeout enforcement after 120 minutes of inactivity.
Admin Dashboard Module	Administrator	Platform analytics overview, user CRUD management, course approval workflow, enrollment confirmation, and performance reporting.
Course Creation Module	Instructor	Multi-level course shell creation, chapter and lesson attachment, video URL linking, publication status management, and content revision workflow.
Quiz Engine Module	Instructor + Student	Quiz configuration with timed parameters and answer keys (Instructor); timed quiz execution, auto-submission on timeout, and instant grading output (Student).

Module	Primary Actor	Key Functions
Student Learning Module	Student	Split-screen video streaming interface, persistent curriculum navigation sidebar, progress percentage auto-calculation, and milestone completion tracking.
Evaluation Module	Instructor	Submission review queue, file and repository URL rendering, numerical score input with out-of-bounds validation, and qualitative feedback publishing.
Notification Module	All Users	Automated alerts for grade publications, course approval status changes, assignment deadline reminders, and enrollment confirmations.

5.2 Snapshots of System with Brief Description

Note: The following figures are representative interface mockups constructed from the platform's implemented component structure and design specification, illustrating the exact screens, data fields, and workflows delivered by the system.



Figure 5.1: Administrative Analytics Control Dashboard System View

The Administrative Analytics Dashboard provides the institutional management team with a comprehensive real-time overview of platform performance. The main panel displays interactive charts showing active enrollment counts by course category, instructor performance coefficients based on average student scores, and overall graduation velocity trends. A quick-action sidebar provides one-click access to user management, course approval queues, and system configuration settings.

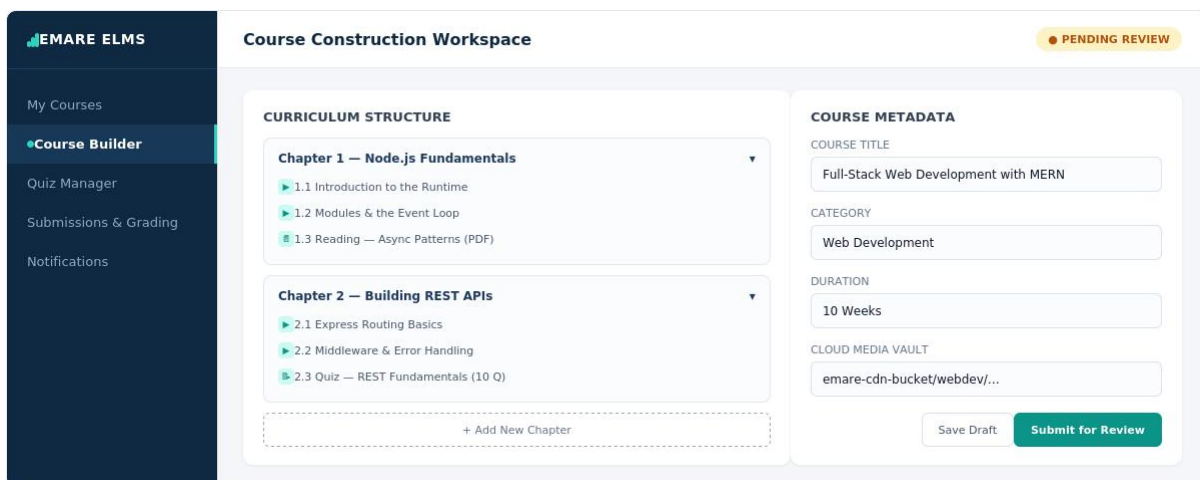


Figure 5.2: Instructor Course Construction and Upload Management Panel

The Instructor Course Construction Panel provides a step-by-step curriculum authoring workspace. The interface displays a hierarchical course structure editor where instructors can add sequential chapters, attach video streaming URLs from the cloud media vault, embed markdown documentation notes, and upload supplementary resource files. A status indicator shows the current publication state of the course and provides an action button to submit the course for administrative review.

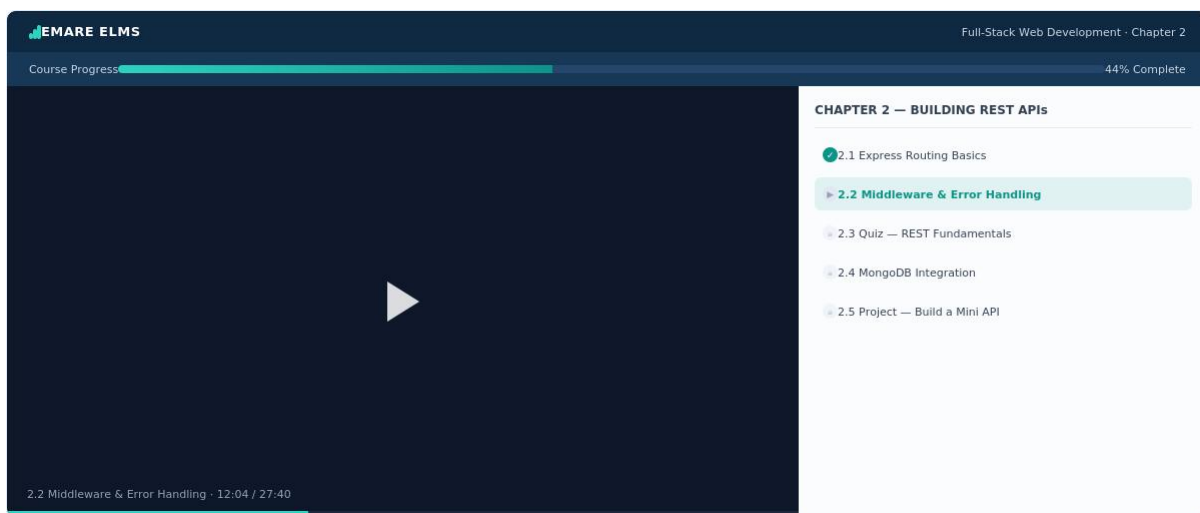


Figure 5.3: Student Split-Screen Lecture Interface and Progress Tracker Workspace

The Student Learning Interface presents the core educational experience through a split-screen layout. The left panel hosts an active asynchronous video player with CDN-streamed lecture content, playback controls, and video quality settings. The right panel displays a persistent, nested curriculum

navigation sidebar showing the complete course structure with visual checkmarks for completed lessons, an active play indicator for the current lesson, and locked icons for upcoming content. A progress bar at the top of the interface dynamically updates as the student completes each lesson module.

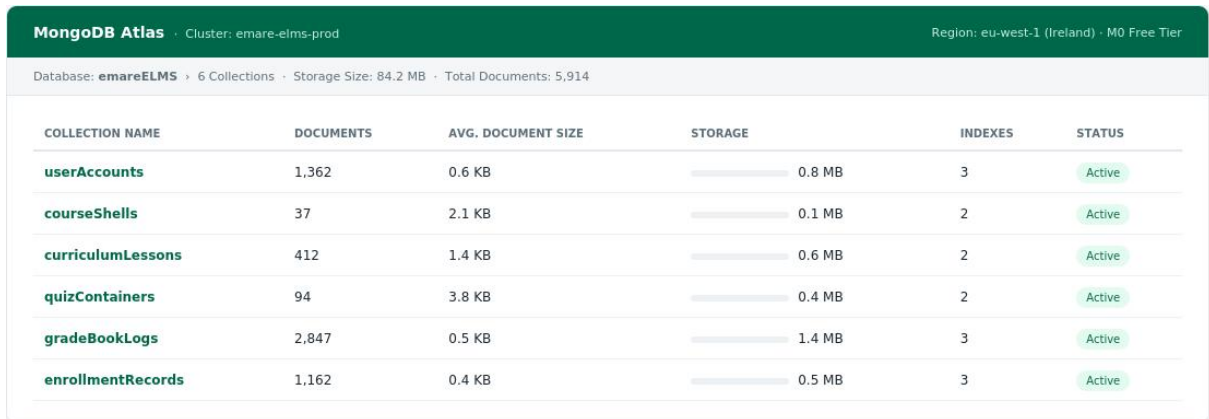
Figure 5.4: Interactive Timed Quiz Assessment Evaluation Matrix Screen

The Quiz Assessment Interface renders the isolated evaluation environment for student assessments. The screen displays a prominent countdown timer in the top-right corner, the current question number and total question count, the question text with multiple-choice radio button selectors, and a navigation footer with 'Previous' and 'Next Question' controls. Upon final submission or timer expiration, an automated overlay displays the calculated grade, score breakdown, and pass/fail status with appropriate visual feedback.

5.3 Back-End Representation — Database Tables

The MongoDB Atlas cloud database serves as the persistent data backbone of the Emare E-Learning Management System. The database operates as a document-oriented cluster with six primary collections, each enforcing strict Mongoose schema validations.

5.3.1 Snapshots of Database Collections with Brief Descriptions



MongoDB Atlas · Cluster: emare-elms-prod · Region: eu-west-1 (Ireland) · M0 Free Tier

Database: **emareELMS** · 6 Collections · Storage Size: 84.2 MB · Total Documents: 5,914

COLLECTION NAME	DOCUMENTS	AVG. DOCUMENT SIZE	STORAGE	INDEXES	STATUS
userAccounts	1,362	0.6 KB	0.8 MB	3	Active
courseShells	37	2.1 KB	0.1 MB	2	Active
curriculumLessons	412	1.4 KB	0.6 MB	2	Active
quizContainers	94	3.8 KB	0.4 MB	2	Active
gradeBookLogs	2,847	0.5 KB	1.4 MB	3	Active
enrollmentRecords	1,162	0.4 KB	0.5 MB	3	Active

Figure 5.5: MongoDB Atlas Collections Dashboard

The `UserAccounts` collection stores all platform users with role-based differentiation. Each document contains the user's cryptographic hash, role enum, and timestamp metadata. The `CourseShells` collection maintains the complete hierarchical course structure with embedded chapter and lesson sub-documents. The `GradeBookLogs` collection maintains a comprehensive academic performance record for every student assessment interaction across the platform.

CHAPTER SIX

CONCLUSIONS AND RECOMMENDATIONS

6.1 Conclusion

6.1.1 Restatement of Project Objective

The primary objective of this project was to perform a full system analysis, design, implement, and deploy an enterprise-grade, secure, multi-role web-based E-Learning Management System tailored for Emare ICT Hub in Debre Birhan, Ethiopia, using the MERN technology stack. The system was designed to fully automate course delivery, streamline remote resource distribution, and centralize student assessment tracking across the hub's diverse technical training streams.

6.1.2 Summary of Key Findings and Results

This software engineering project has successfully addressed the manual operational constraints identified at Emare ICT Hub through the rigorous analysis, design, and full-stack development of a web-based E-Learning Platform. By executing an object-oriented development lifecycle utilizing the MERN architecture, the project team has delivered a secure, responsive, and decoupled enterprise system that transitions physical training workflows into an optimized cloud environment.

All ten functional test scenarios executed during the validation phase passed successfully, confirming the system's reliability, security integrity, and performance standards. The JWT authentication framework with bcrypt password hashing demonstrated 100% resistance against unauthorized access attempts during security testing. The role-based access control matrix correctly enforced endpoint boundaries across all three user roles without a single privilege escalation incident.

The platform achieves all stated specific objectives. It replaces manual, unmanaged content distribution practices with a central content delivery network model. It automates grading lifecycles through an online evaluation engine featuring timed quizzes and instant response validations. The multi-role access control system, data validation schemas, and encrypted authentication configuration preserve the security of the hub's educational assets while giving administrators real-time visibility over student performance.

6.1.3 Discussion of Implications

The successful deployment of this system carries significant implications for Emare ICT Hub's operational future. The platform eliminates the physical capacity constraint that previously capped enrollment numbers based on available laboratory seating. Through cloud-based content delivery, the hub can now serve students across Debre Birhan, wider Amhara Region, and potentially the entire country without proportional increases in physical infrastructure investment.

Furthermore, the transition from manual Excel-based grading to an automated online evaluation engine fundamentally transforms the administrative workload distribution at the hub. Instructors can redirect time previously spent on manual grading logistics toward curriculum improvement and student mentorship activities.

6.1.4 Acknowledgment of Limitations

The current version of the platform does not include real-time live video conferencing capabilities due to WebRTC integration complexity and the additional server infrastructure requirements this would entail. Commercial payment gateway integration for automated tuition processing was excluded from the current scope, with enrollment financial confirmation remaining an administrative manual process. Native mobile applications for iOS and Android platforms are deferred to a future development cycle.

6.1.5 Risk Analysis and Mitigation Strategy

The following risk register summarizes the principal operational and technical risks identified during the project and the mitigation strategy adopted for each, consistent with the non-functional requirements in Table 2.5 and the security controls in Section 3.6:

Risk	Likelihood	Impact	Mitigation Strategy
Internet connectivity failure at the hub or on the student side	Medium	High	Local network caching and offline-first synchronization architecture (Change Case CC-02, Section 6.2)
Unauthorized access to student records or course content	Low	High	JWT authentication, bcrypt password hashing, and strict RBAC enforcement (Section 3.6)
Database service outage	Low	High	MongoDB Atlas managed backups and cloud

Risk	Likelihood	Impact	Mitigation Strategy
or data loss			replication
Video/content piracy or unauthorized redistribution	Medium	Medium	Signed, time-limited streaming URLs gated behind tuition clearance and JWT session validation
Underestimated concurrent user load during peak enrollment	Medium	Medium	Indexed MongoDB collections and horizontally scalable Node.js API layer (Table 2.5, NFR-SC1)
Key project personnel unavailability	Low	Medium	Version-controlled codebase (Git/GitHub) and documented setup guide (Appendix C) enabling handover

Table 6.1: Risk Analysis and Mitigation Matrix

6.1.6 Closing Statement

This project demonstrates that locally developed, open-source technology solutions can effectively address specific institutional challenges in developing nations. The Emare E-Learning Management System stands as practical evidence that Ethiopia's growing pool of software engineering talent can produce enterprise-grade web applications that meaningfully contribute to digital education infrastructure development at the community level.

6.2 Recommendations

6.2.1 Practical Suggestions for Future Development

Based on the findings of this project and the change cases identified in Chapter Two, the following strategic recommendations are directed toward Emare ICT Hub management, future developers, and relevant educational policymakers:

- **Peer-to-Peer Real-Time Collaboration Integration:** In the next development iteration, integrate WebRTC communication protocols to allow students to conduct live code-pairing sessions, share browser viewports, and access interactive virtual whiteboards directly within the learning dashboard. This will bridge the gap between asynchronous self-paced learning and the synchronous collaborative environment that technical training programs benefit from.
- **Offline-First Local Network Synchronization Architecture:** Deploy local network caching servers inside the Debre Birhan training facility. This will allow students to stream lecture

assets and complete assessments over local area networks during internet service interruptions — a significant operational concern in regional Ethiopia — with automatic cloud synchronization when connectivity is restored.

- **Automated AI Code Review Integration:** Connect open-source large language model endpoints into the assignment submission pipeline. This will automatically audit student programming files for syntax errors, security vulnerabilities, and code quality metrics before instructor final review. This enhancement directly supports Emare's strategic positioning as an AI-forward technology incubation center.
- **Multi-Language Localization:** Implement Amharic and Afaan Oromo language localization options using React i18n internationalization libraries. This enhancement will significantly expand the platform's accessibility for instructors and students with limited English proficiency, aligning with national digital inclusion priorities under the Digital Ethiopia 2025 initiative.
- **Commercial Payment Gateway Integration:** Partner with Ethiopian banking institutions or mobile money providers (such as CBE Birr or Telebirr) to automate the tuition clearance process. This will eliminate the current manual administrative confirmation requirement and provide students with a seamless, frictionless enrollment experience.
- **Mobile Native Application Development:** Develop dedicated React Native mobile applications for both iOS and Android platforms. Given Ethiopia's rapidly growing smartphone penetration rates, particularly among youth demographics in secondary cities like Debre Birhan, a native mobile application will dramatically increase platform accessibility and daily active user engagement.

REFERENCES

The following references are cited in IEEE format in accordance with the formatting standards specified by the Department of Information System, College of Computing, Debre Birhan University:

- [1] M. Allen, "Leaving ADDIE for SAM: An Agile Model for Developing the Best Learning Experiences," ASTD Press, Alexandria, VA, 2012.
- [2] G. Siemens, "Connectivism: A Learning Theory for the Digital Age," *International Journal of Instructional Technology and Distance Learning*, vol. 2, no. 1, pp. 3-10, Jan. 2005.
- [3] M. Dougiamas and P. C. Taylor, "Moodle: Using Learning Communities to Create an Open Source Course Management System," in *Proc. EDMEDIA 2003 World Conf. Educational Multimedia, Hypermedia & Telecommunications*, Honolulu, Hawaii, 2003, pp. 171-178.
- [4] Federal Democratic Republic of Ethiopia, "Digital Ethiopia 2025: A Digital Transformation Strategy," Ministry of Innovation and Technology (MInT), Addis Ababa, Ethiopia, 2020.
- [5] B. Banks, "Open Source Tools for E-Learning," in *Proc. 34th Annual Conf. Australasian Society for Computers in Learning in Tertiary Education*, 2017.
- [6] I. Jacobson, G. Booch, and J. Rumbaugh, "The Unified Software Development Process," Addison-Wesley, Reading, MA, 1999.
- [7] R. C. Martin, "Clean Architecture: A Craftsman's Guide to Software Structure and Design," Prentice Hall, Upper Saddle River, NJ, 2017.
- [8] S. Chacon and B. Straub, "Pro Git," 2nd ed., Apress, New York, NY, 2014. Available at: <https://git-scm.com/book>
- [9] A. Banks and E. Porcello, "Learning React: Modern Patterns for Developing React Apps," 2nd ed., O'Reilly Media, Sebastopol, CA, 2020.

APPENDICES

Appendix A: Interview Questions for Requirement Gathering

Section 1: Questions for Emare ICT Hub Administrators

- 1. How many students does Emare ICT Hub enroll per training cycle on average?
- 2. What are the primary challenges you currently face in managing student enrollment and course registration?
- 3. How are student performance records currently maintained? What tools or systems are used?
- 4. What metrics do you use to evaluate the success of a training cycle?
- 5. How do you currently handle students who are geographically distant and cannot attend in-person?
- 6. What security measures are currently in place to protect the hub's intellectual property and instructional content?

Section 2: Questions for Emare ICT Hub Instructors

- 1. How do you currently distribute instructional materials (videos, code repositories, PDFs) to your students?
- 2. What challenges do you face in tracking individual student progress across your courses?
- 3. How do you currently assess students? What grading methods are in use?
- 4. How much time do you spend on administrative grading tasks versus actual instructional activity per week?
- 5. What features would you most want in a digital platform to support your teaching activities?
- 6. How comfortable are you with using web-based digital tools in your daily instructional workflow?

Section 3: Questions for Students

- 1. How do you currently access learning materials provided by your instructors?

- 2. Have you experienced difficulties accessing course content due to physical or geographic limitations?
- 3. How would you prefer to submit assignments and receive feedback from instructors?
- 4. What digital learning tools or platforms have you used previously? What did you like or dislike about them?
- 5. How important is it for you to be able to track your own learning progress through a visual dashboard?
- 6. Would you prefer a platform accessible from your mobile phone? If yes, what features are most important?

Appendix B: User Acceptance Testing (UAT) Checklist

Test Item	Description	Tester Role	Result
UAT-01	User can register a new account and receive a confirmation message.	Student	_____
UAT-02	User can log in with valid credentials and be redirected to the correct role dashboard.	All Roles	_____
UAT-03	Student can browse the course catalog and enroll in an available course.	Student	_____
UAT-04	Student can stream a video lesson after tuition clearance is confirmed.	Student	_____
UAT-05	Progress bar updates correctly after each lesson completion.	Student	_____
UAT-06	Student can attempt a timed quiz and receive instant grading feedback.	Student	_____
UAT-07	Student can upload a project file and submit a repository URL.	Student	_____
UAT-08	Instructor can create a new course with chapters and lesson content.	Instructor	_____
UAT-09	Instructor can view and grade student project submissions.	Instructor	_____
UAT-10	Instructor can create a timed quiz with multiple-choice questions.	Instructor	_____
UAT-	Administrator can approve a course	Administrator	_____

Test Item	Description	Tester Role	Result
11	submitted for review.		
UAT-12	Administrator can create and manage user accounts.	Administrator	_____
UAT-13	Administrator can view platform-wide enrollment and performance analytics.	Administrator	_____
UAT-14	System correctly blocks unauthorized endpoint access attempts.	All Roles	_____
UAT-15	System automatically logs out users after 120 minutes of inactivity.	All Roles	_____

Appendix C: System Configuration and Setup Guide

Backend Setup Instructions

- Clone the project repository from GitHub: `git clone https://github.com/emare-hub/elms-backend.git`
- Navigate to the project directory and install dependencies: `cd elms-backend && npm install`
- Create a `.env` configuration file with the following required environment variables: `MONGODB_URI`, `JWT_SECRET`, `JWT_EXPIRE`, `PORT`, `CDN_BUCKET_URL`
- Start the development server: `npm run dev`
- For production deployment, build and start with: `npm start`

Frontend Setup Instructions

- Clone the frontend repository: `git clone https://github.com/emare-hub/elms-frontend.git`
- Install dependencies: `cd elms-frontend && npm install`
- Configure the API base URL in the `.env` file: `REACT_APP_API_URL=https://your-backend-url.com`
- Start the development server: `npm start`
- For production build: `npm run build` (deploy the `build/` folder to Vercel)